

您的第一個 Flutter 應用程式

來源：<https://codelabs.developers.google.com/codelabs/flutter-codelab-first?hl=zh-tw>

步驟數：9

在本程式碼研究室中，您將學習如何建構 Flutter 應用程式，這個應用程式可產生隨機、好聽的名稱。

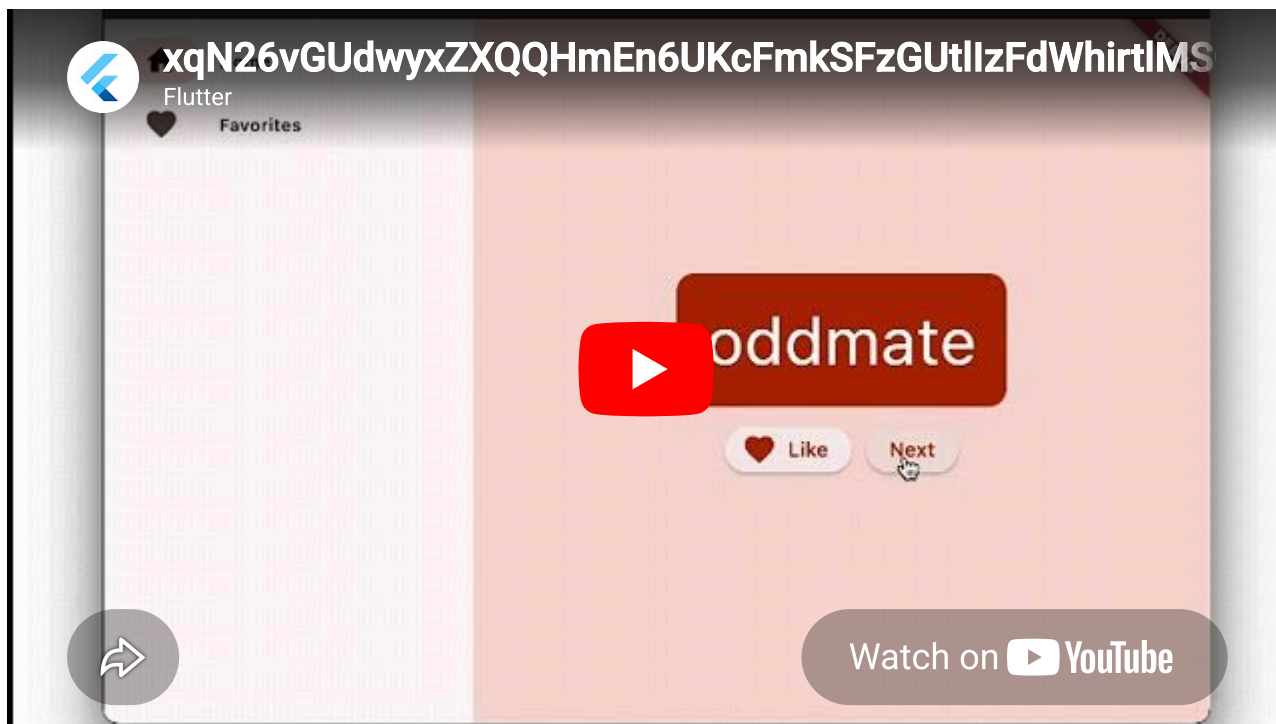
目錄

1. [1. 簡介](#)
2. [2. 設定 Flutter 環境](#)
3. [3. 建立專案](#)
4. [4. 新增按鈕](#)
5. [5. 讓應用程式更美觀](#)
6. [6. 新增功能](#)
7. [7. 新增導覽邊欄](#)
8. [8. 新增網頁](#)
9. [9. 後續步驟](#)

步驟 1 · 約 0 分鐘

1. 簡介

Flutter 是 Google 的 UI 工具包，可讓您根據單一程式碼集，建構適用於行動裝置、網頁和電腦的應用程式。在本程式碼研究室中，您將建構下列 Flutter 應用程式：

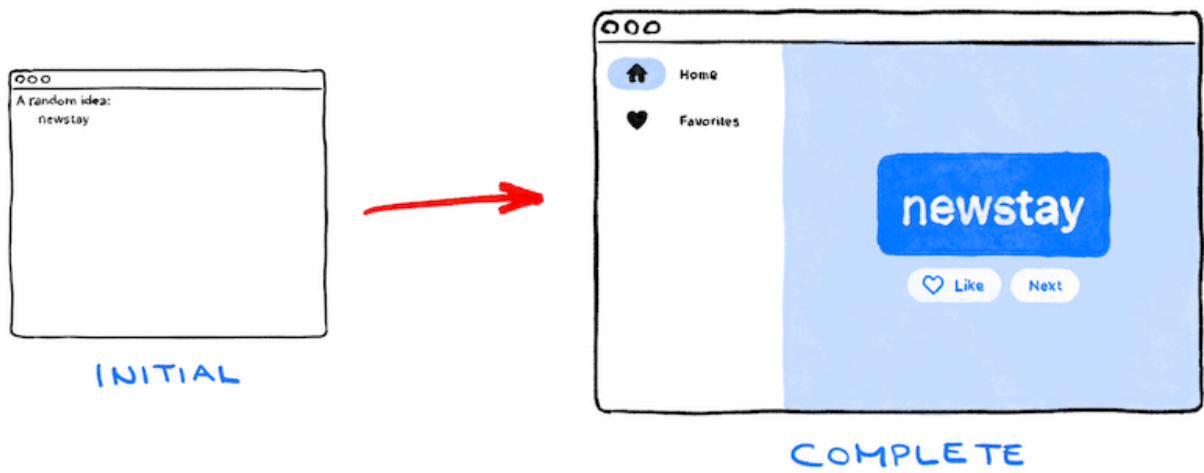


應用程式會產生聽起來很酷的名稱，例如「newstay」、「lightstream」、「mainbrake」或「graypine」。使用者可以要求下一個名稱、將目前的名稱加入我的最愛，以及在另一個頁面查看我的最愛名稱清單。應用程式可配合不同螢幕大小調整。

課程內容

- Flutter 的基本運作方式
- 在 Flutter 中建立版面配置
- 將使用者互動 (例如按下按鈕) 連結至應用程式行為
- 讓 Flutter 程式碼井然有序
- 讓應用程式可配合不同螢幕調整
- 打造應用程式一致的外觀和風格

我們會先提供基本架構，讓您直接跳到有趣的部分。



Filip 將帶領您完成整個程式碼研究室！



按一下「下一步」即可開始實驗室。

步驟 2 · 約 10 分鐘

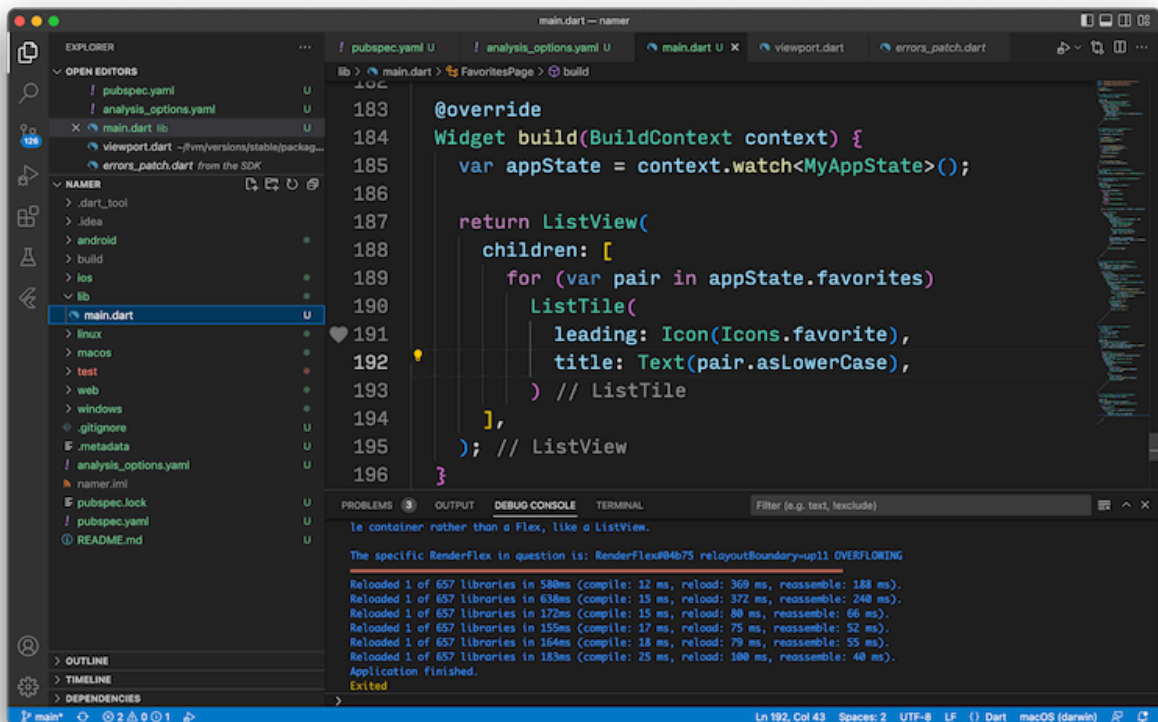
2. 設定 Flutter 環境

編輯者

為盡量簡化本程式碼研究室，我們假設您會使用 **Visual Studio Code** (VS Code) 做為開發環境。這項服務免費提供，且支援所有主要平台。

當然，您可以使用任何喜歡的編輯器，例如 Android Studio、其他 IntelliJ IDE、Emacs、Vim 或 Notepad++，這些編輯器都支援 Flutter。

本程式碼研究室建議使用 VS Code，因為操作說明預設會使用 VS Code 專屬的快速鍵。例如，與其說「在編輯器中執行適當動作來完成 X」，不如說「按這裡」或「按下這個鍵」。

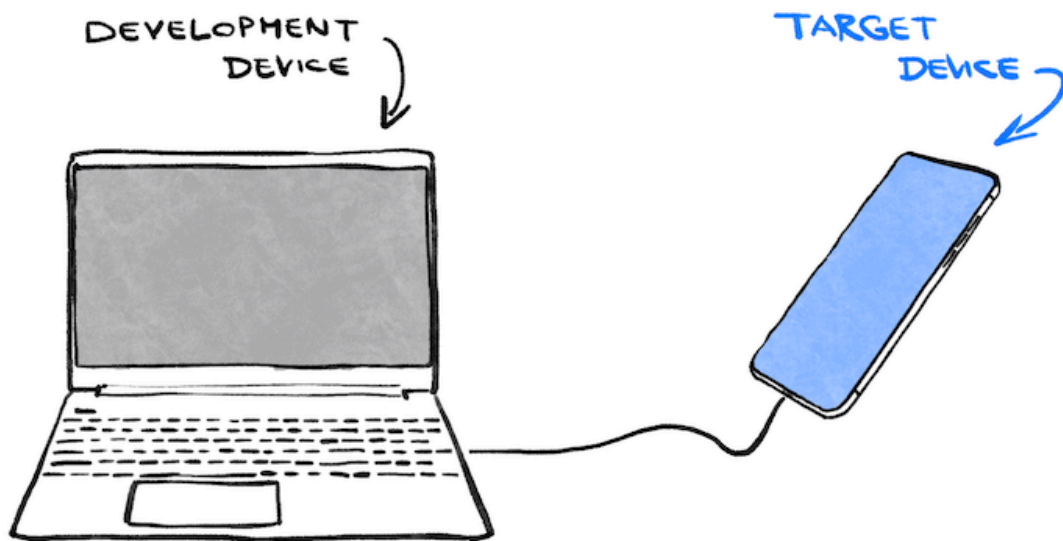


選擇開發目標

Flutter 是跨平台工具包，您的應用程式可以在下列任一作業系統上執行：

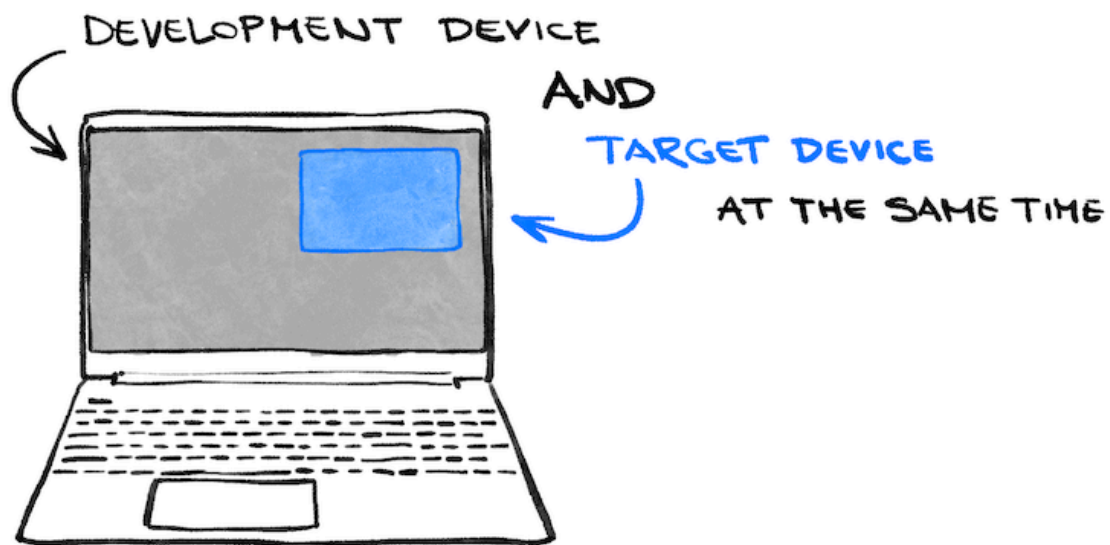
- iOS
- Android
- Windows
- macOS
- Linux
- 網路

不過，一般來說，您會選擇主要開發時使用的單一作業系統。這個作業系統就是「開發目標」：開發過程中用來執行應用程式的 OS。



舉例來說，假設您使用 Windows 筆電開發 Flutter 應用程式。如果您選擇 Android 做為開發目標，通常會使用 USB 傳輸線將 Android 裝置連接至 Windows 筆電，然後在該 Android 裝置上執行開發中的應用程式。但您也可以選擇 Windows 做為開發目標，這表示開發中的應用程式會與編輯器一起以 Windows 應用程式的形式執行。

提示：強烈建議選擇開發裝置的作業系統做為開發目標。舉例來說，如果您的電腦執行 Windows，請選擇 Windows 做為開發目標。



您可能會想選取網頁做為開發目標，但這麼做會導致您無法使用 Flutter 最實用的開發功能之一：有狀態熱重載。Flutter 無法熱重載網頁應用程式。

請立即選擇。請注意：您之後隨時可以在其他作業系統上執行應用程式。只是心中有明確的開發目標，後續步驟會更順利。

安裝 Flutter

如需 Flutter SDK 的最新安裝說明，請前往 docs.flutter.dev。

Flutter 網站上的操作說明不僅涵蓋 SDK 本身的安裝作業，也包括開發目標相關工具和編輯器外掛程式。請注意，在本程式碼研究室中，您只需要安裝下列項目：

1. Flutter SDK
2. 安裝 Flutter 外掛程式的 Visual Studio Code
3. 所選開發目標所需的軟體 (例如：以 Windows 為目標的 *Visual Studio*，或以 macOS 為目標的 *Xcode*)

在下一節中，您將建立第一個 Flutter 專案。

如果目前遇到問題，您或許可以參考 StackOverflow 上的這些問答內容，進行疑難排解。

常見問題

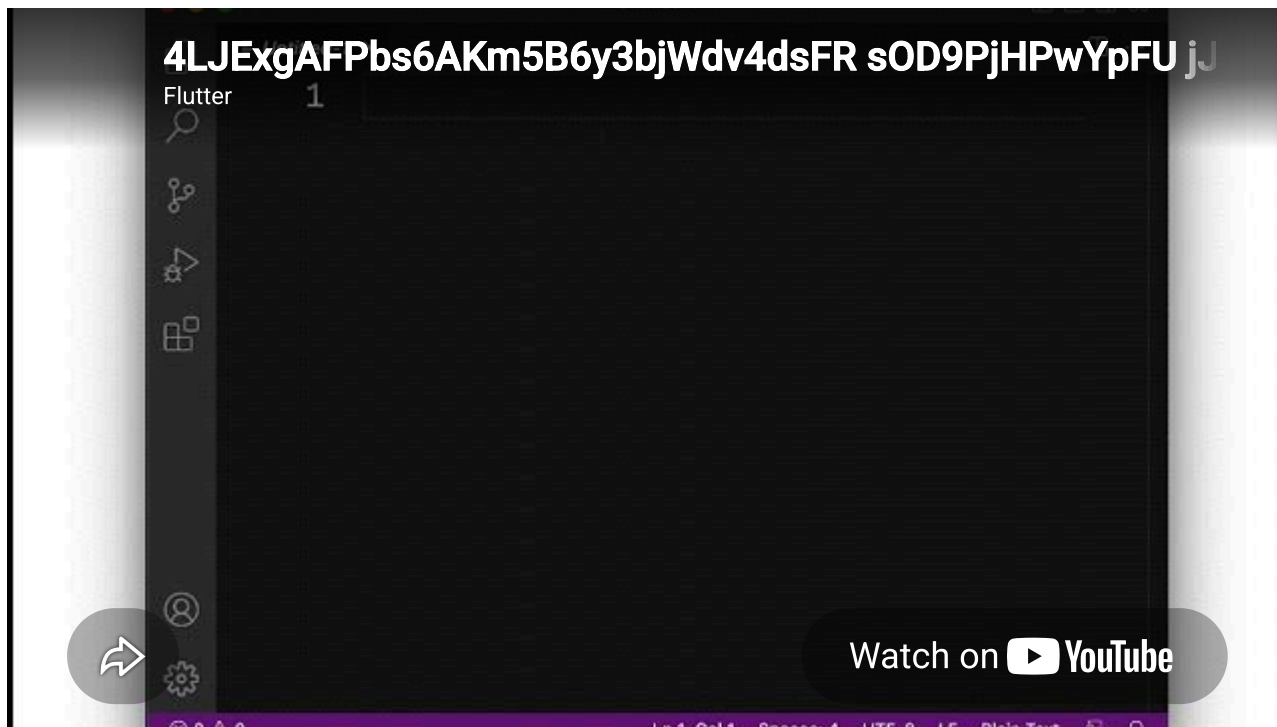
- [如何找出 Flutter SDK 的路徑？](#)
 - [如果找不到 Flutter 指令，該怎麼辦？](#)
 - [如何修正「Waiting for another flutter command to release the startup lock」問題？](#)
 - [如何告知 Flutter Android SDK 的安裝位置？](#)
 - 執行 `flutter doctor --android-licenses` 時發生 Java 錯誤，該如何處理？
 - 如何解決找不到 Android `sdkmanager` 工具的問題？
 - 如何處理「缺少 `cmdline-tools` 元件」錯誤？
 - [如何在 Apple 晶片 \(M1\) 上執行 CocoaPods？](#)
 - [如何在 VS Code 中停用儲存時自動格式化功能？](#)
-

步驟 3 · 約 5 分鐘

3. 建立專案

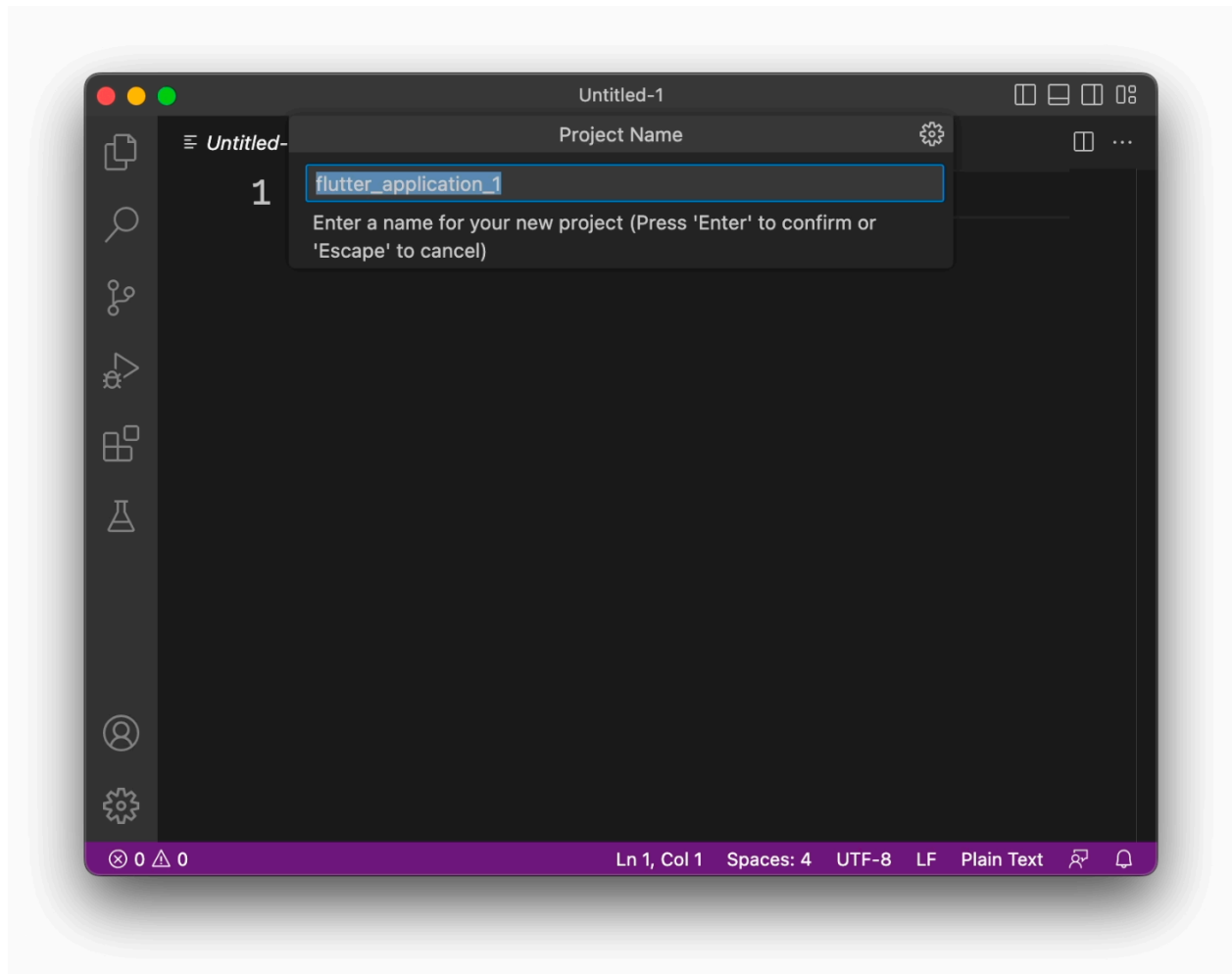
建立第一個 Flutter 專案

啟動 Visual Studio Code 並開啟指令面板 (按下 `F1`、`Ctrl+Shift+P` 或 `Shift+Cmd+P` 鍵)。開始輸入「flutter new」。選取「Flutter: New Project」指令。



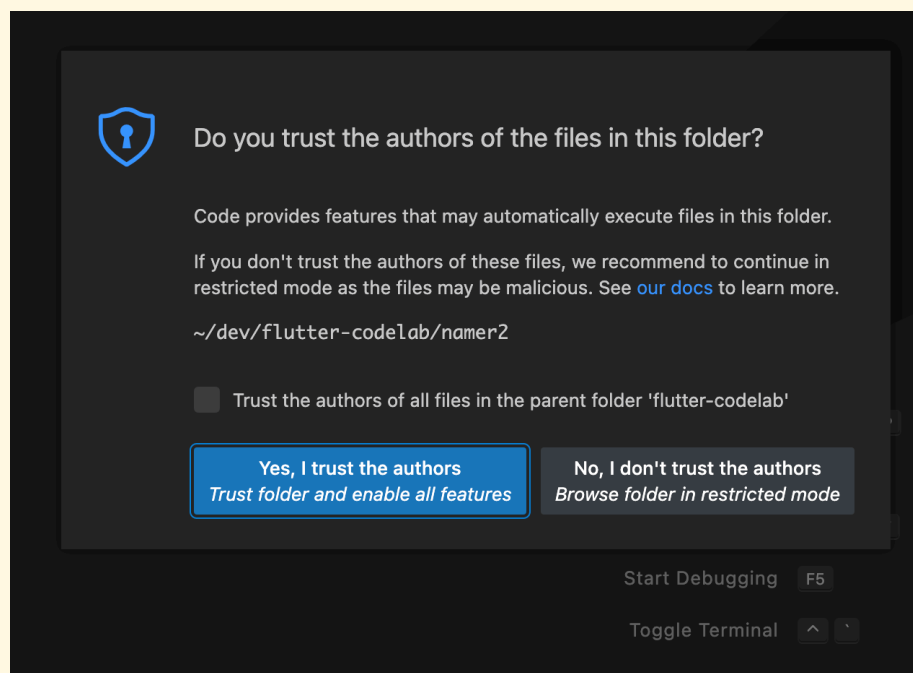
接著，選取「應用程式」，然後選取要建立專案的資料夾。這可能是主目錄，或類似 `C:\src\` 的內容。

最後，請為專案命名。例如 `namer_app` 或 `my_awesome_namer`。



Flutter 會建立專案資料夾，並在 VS Code 中開啟。

注意：VS Code 會顯示模式視窗，詢問是否信任資料夾內容。

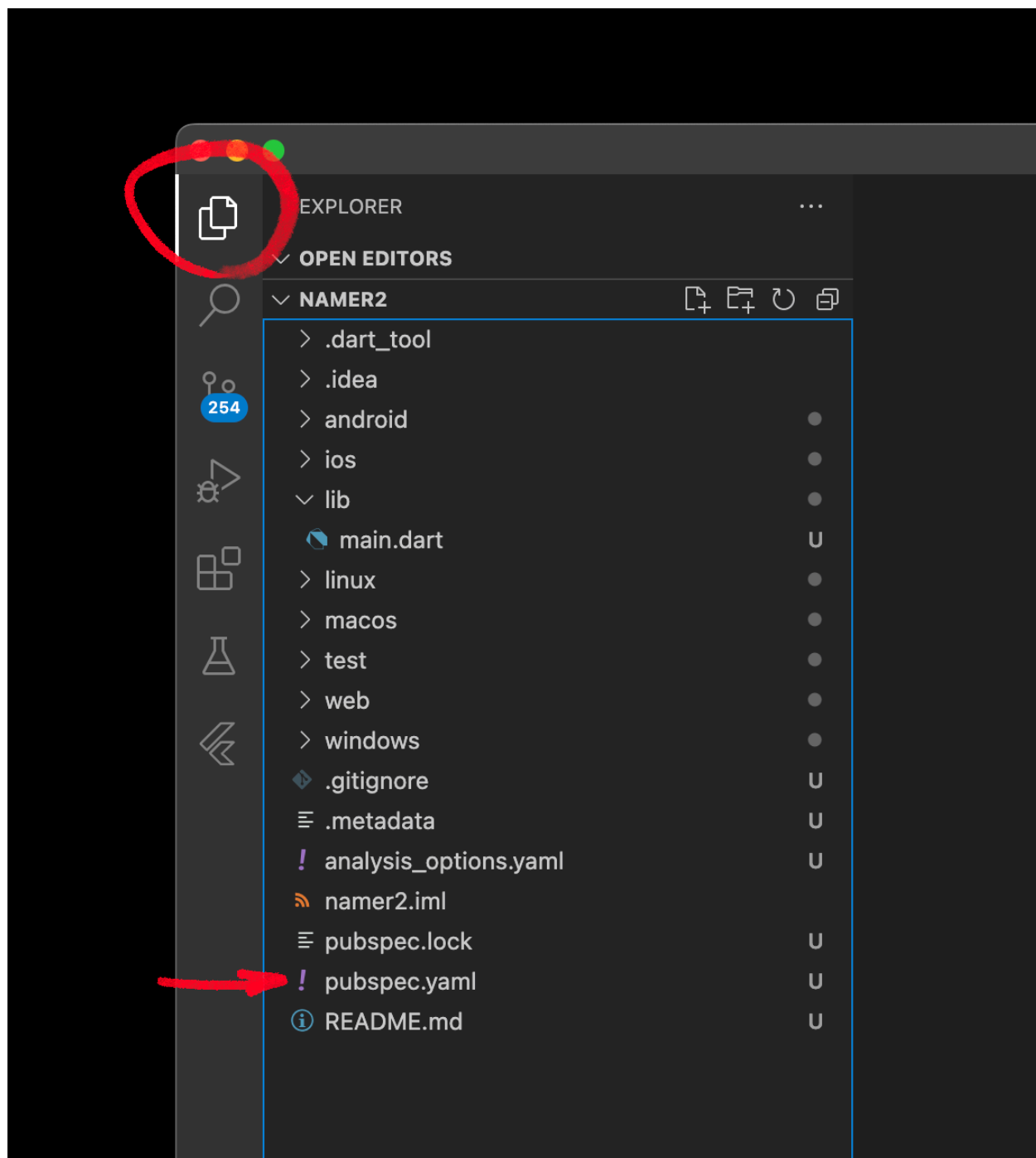


選取「是」，另一個選項會停用 VS Code 中的許多 Flutter 功能。

現在請使用應用程式的基本架構，覆寫 3 個檔案的內容。

複製並貼上初始應用程式

在 VS Code 的左側窗格中，確認已選取「Explorer」，然後開啟 `pubspec.yaml` 檔案。



將這個檔案的內容替換成以下內容：

pubspec.yaml

```
name: namer_app
description: "A new Flutter project."
publish_to: "none"
version: 0.1.0

environment:
  sdk: ^3.9.0

dependencies:
  flutter:
    sdk: flutter
  english_words: ^4.0.0
  provider: ^6.1.5

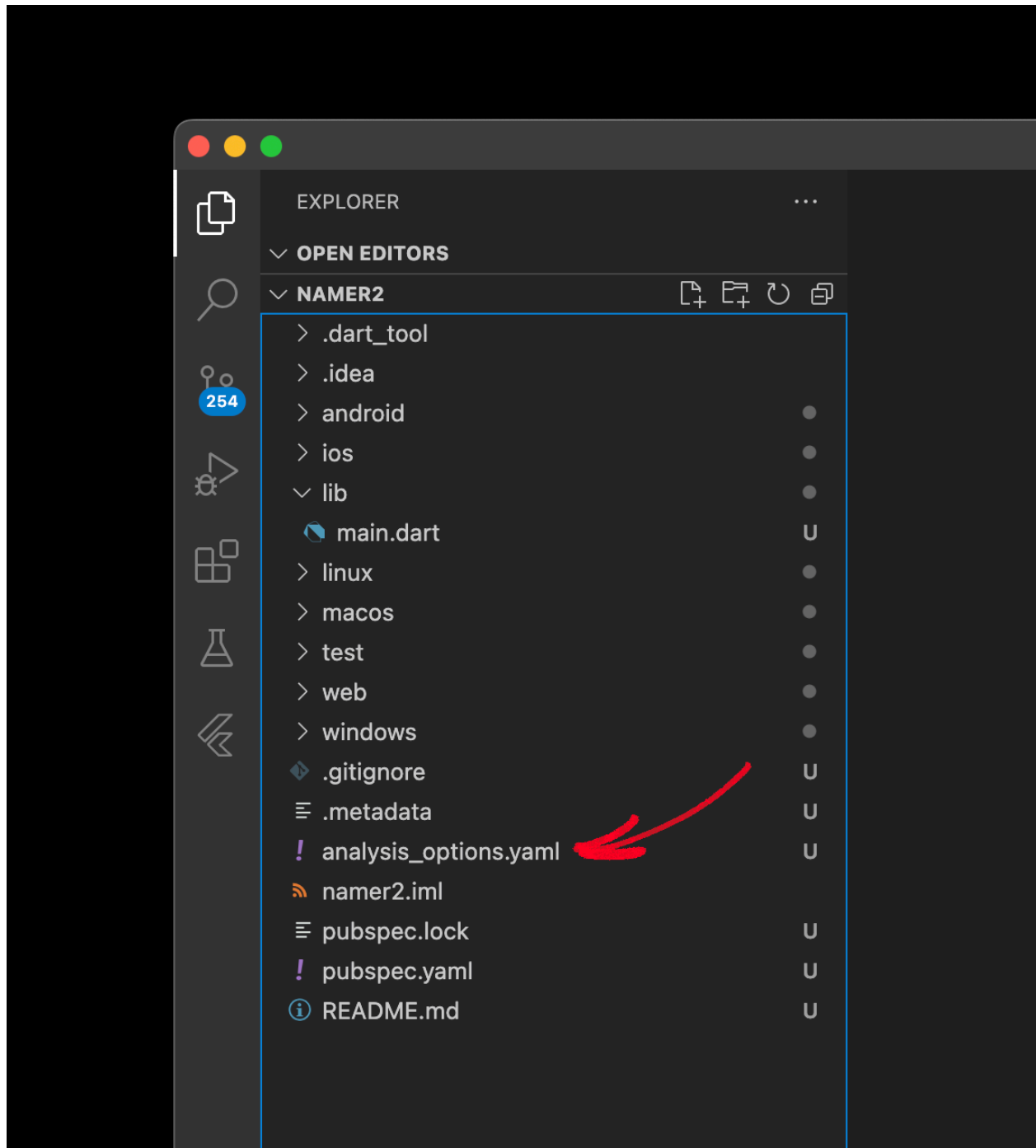
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^6.0.0

flutter:
  uses-material-design: true
```

`pubspec.yaml` 檔案會指定應用程式的基本資訊，例如目前版本、依附元件，以及隨附的資產。

注意：如果應用程式的名稱不是 `namer_app`，請一併變更第一行。

接著，開啟專案中的另一個設定檔 `analysis_options.yaml`。



將其內容換成下列內容：

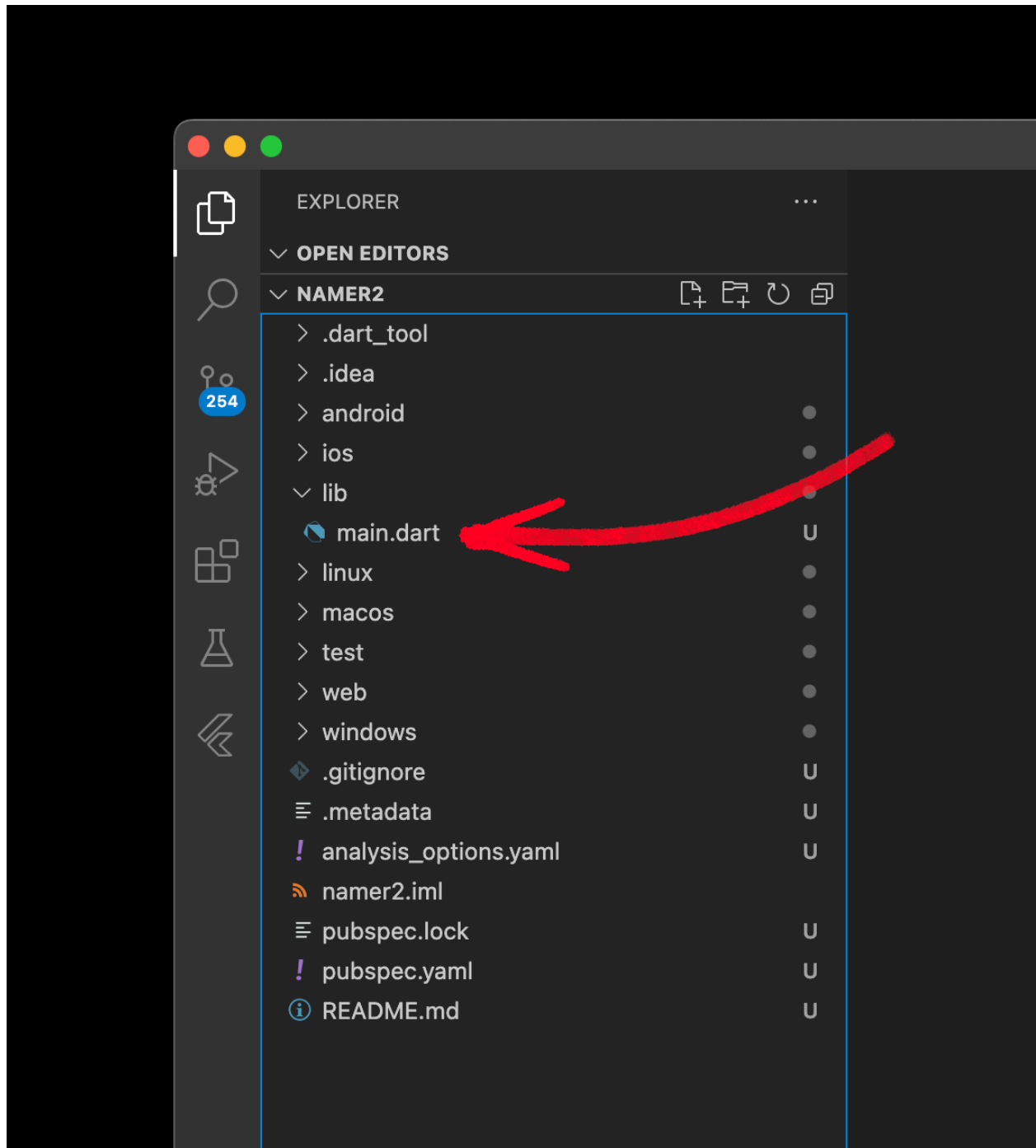
analysis_options.yaml

```
include: package:flutter_lints/flutter.yaml

linter:
  rules:
    avoid_print: false
    prefer_const_constructors_in_immutables: false
    prefer_const_constructors: false
    prefer_const_literals_to_create_immutables: false
    prefer_final_fields: false
    unnecessary_breaks: true
    use_key_in_widget_constructors: false
```

這個檔案會決定 Flutter 分析程式碼時的嚴格程度。由於這是您第一次使用 Flutter，因此您要告知分析器放寬標準。你之後隨時可以調整這項設定。事實上，當您即將發布實際的正式版應用程式時，幾乎一定會希望分析器比這個更嚴格。

最後，開啟 `lib/` 目錄下的 `main.dart` 檔案。



將這個檔案的內容替換成以下內容：

lib/main.dart

```

import 'package:english_words/english_words.dart';
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

void main() {
  runApp(MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => MyAppState(),
      child: MaterialApp(
        title: 'Namer App',
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
        ),
        home: MyHomePage(),
      ),
    );
  }
}

class MyAppState extends ChangeNotifier {
  var current = WordPair.random();
}

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();

    return Scaffold(
      body: Column(
        children: [Text('A random idea:'), Text(appState.current.asLowerCase)],
      ),
    );
  }
}

```

目前為止，這個應用程式的程式碼總共只有 50 行。

注意：本程式碼研究室會快速帶您瞭解如何開始以互動方式變更應用程式，因為這是學習 Flutter 的最佳方式。如果上述程式碼的說明不完整，請耐心等待，我們即將提供。

在下一節中，您將在偵錯模式下執行應用程式，並開始開發。

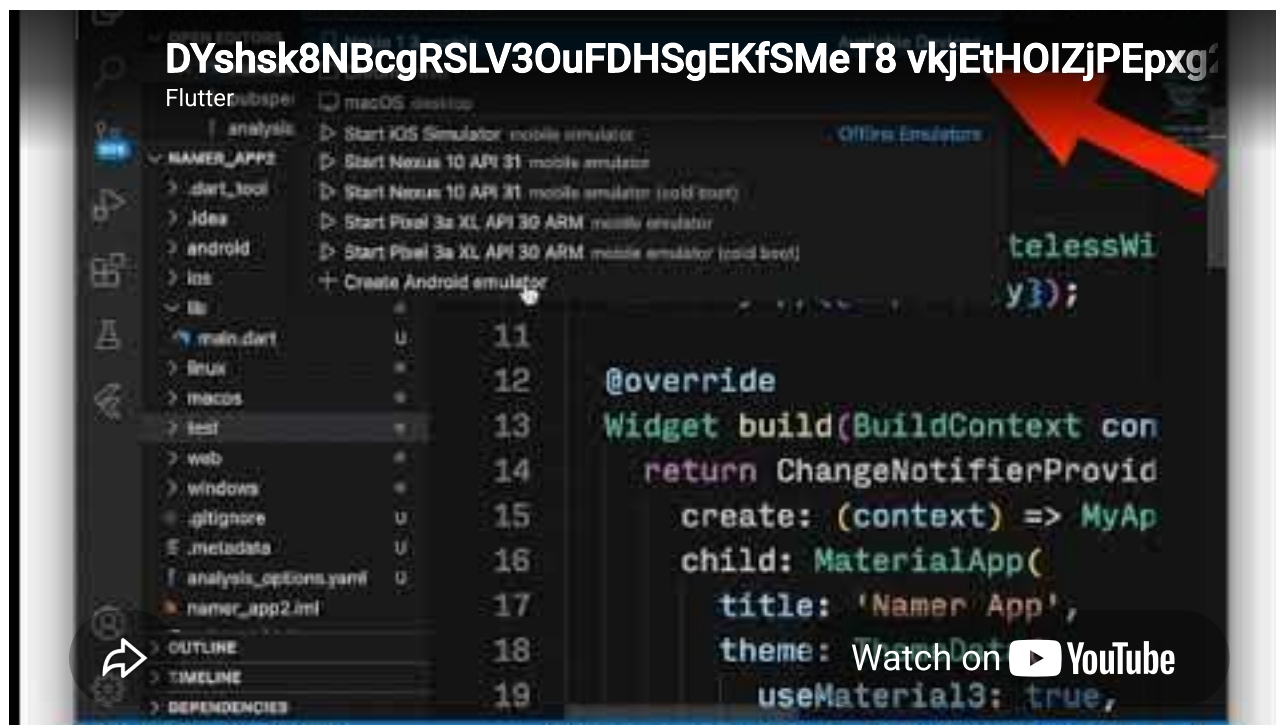
步驟 4 · 約 15 分鐘

4. 新增按鈕

這個步驟會新增「下一步」按鈕，用於產生新的字詞配對。

啟動應用程式

首先，開啟 `lib/main.dart` 並確認已選取目標裝置。在 VS Code 的右下角，你會看到顯示目前目標裝置的按鈕。按一下即可變更。



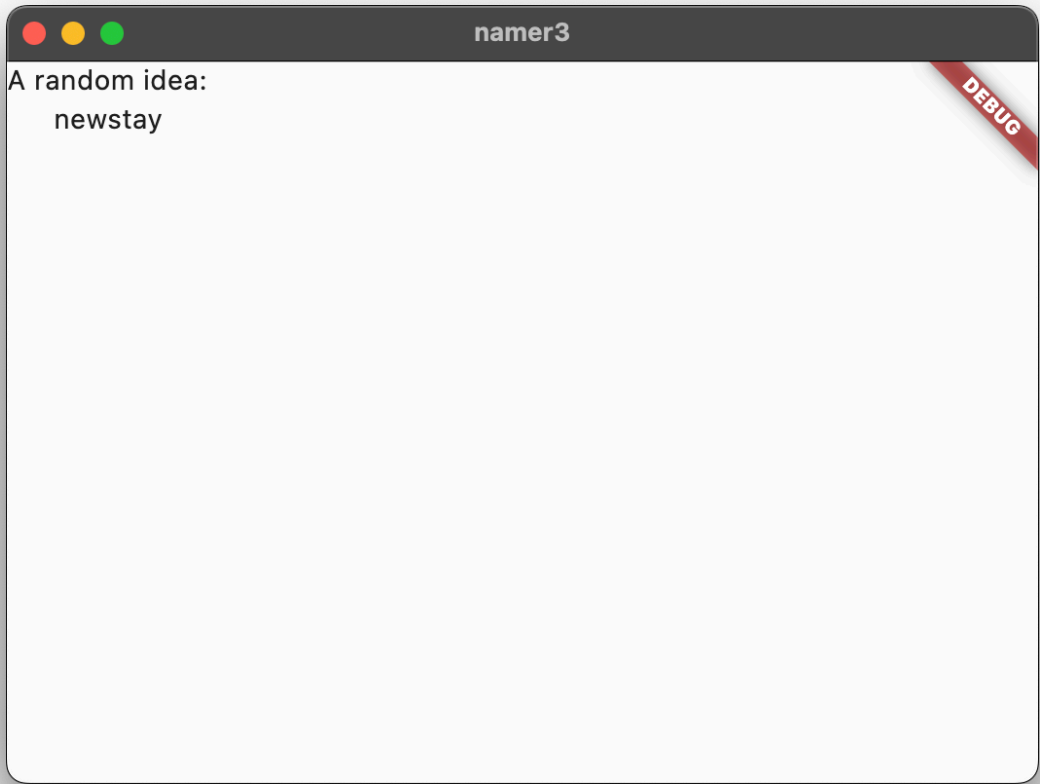
開啟 `lib/main.dart` 後，找出 VS Code 視窗右上角的「播放」



按鈕，然後按一下。



大約一分鐘後，應用程式就會以偵錯模式啟動。目前看起來還不多：



注意：如果開發目標是行動裝置，文字可能會部分遭到凹口或狀態列遮蔽。我們稍後會說明如何修正這個問題。

首次熱重載

在 `lib/main.dart` 底部，將內容新增至第一個 `Text` 物件中的字串，然後儲存檔案 (使用 `Ctrl+S` 或 `Cmd+S`)。例如：

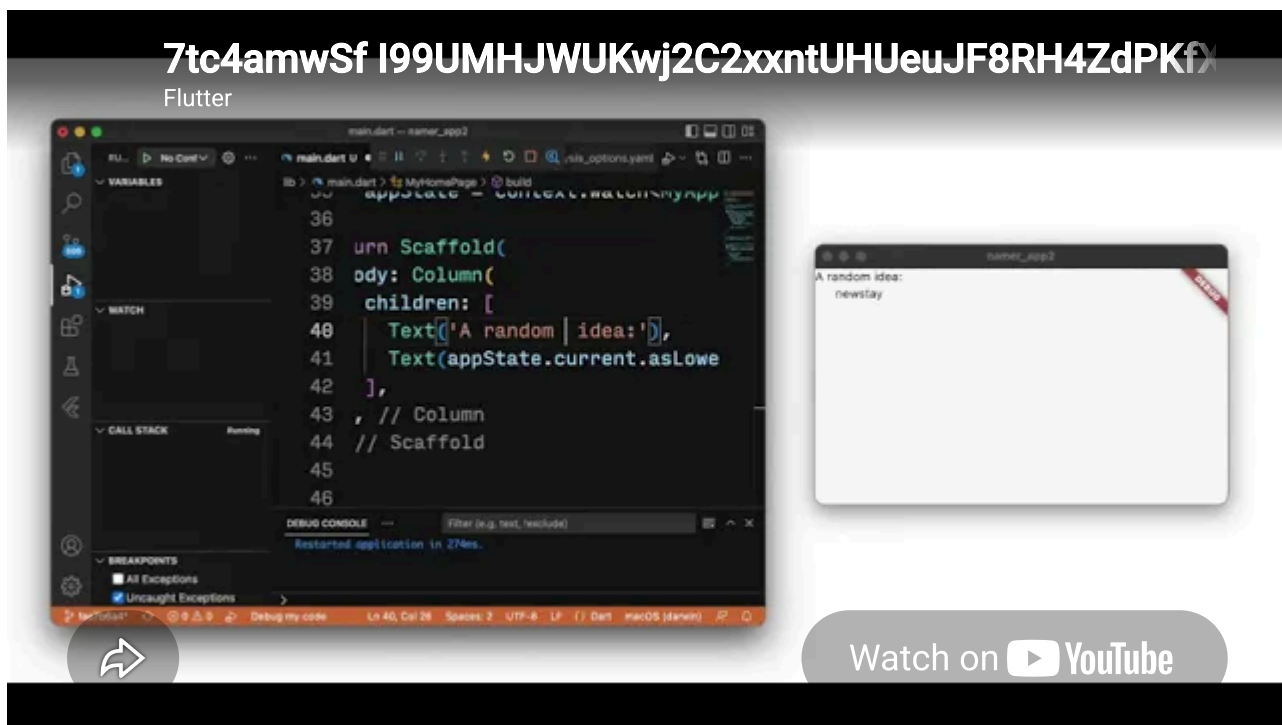
`lib/main.dart`

```
// ...

return Scaffold(
  body: Column(
    children: [
      Text('A random AWESOME idea:'), // ← Example change.
      Text(appState.current.asLowerCase),
    ],
  ),
);

// ...
```

請注意，應用程式會立即變更，但隨機字詞維持不變。這就是 Flutter 著名的有狀態熱重載功能。儲存來源檔案的變更時，系統會觸發熱重載。



常見問題

- 如果 VSCode 無法使用熱重載功能，該怎麼辦？
- 我是否必須在 VSCode 中按下「r」才能進行熱重載？
- 網頁是否支援熱重載？
- 如何移除「Debug」橫幅？

新增按鈕

接著在 `Column` 底部新增按鈕，也就是第二個 `Text` 執行個體正下方。

lib/main.dart

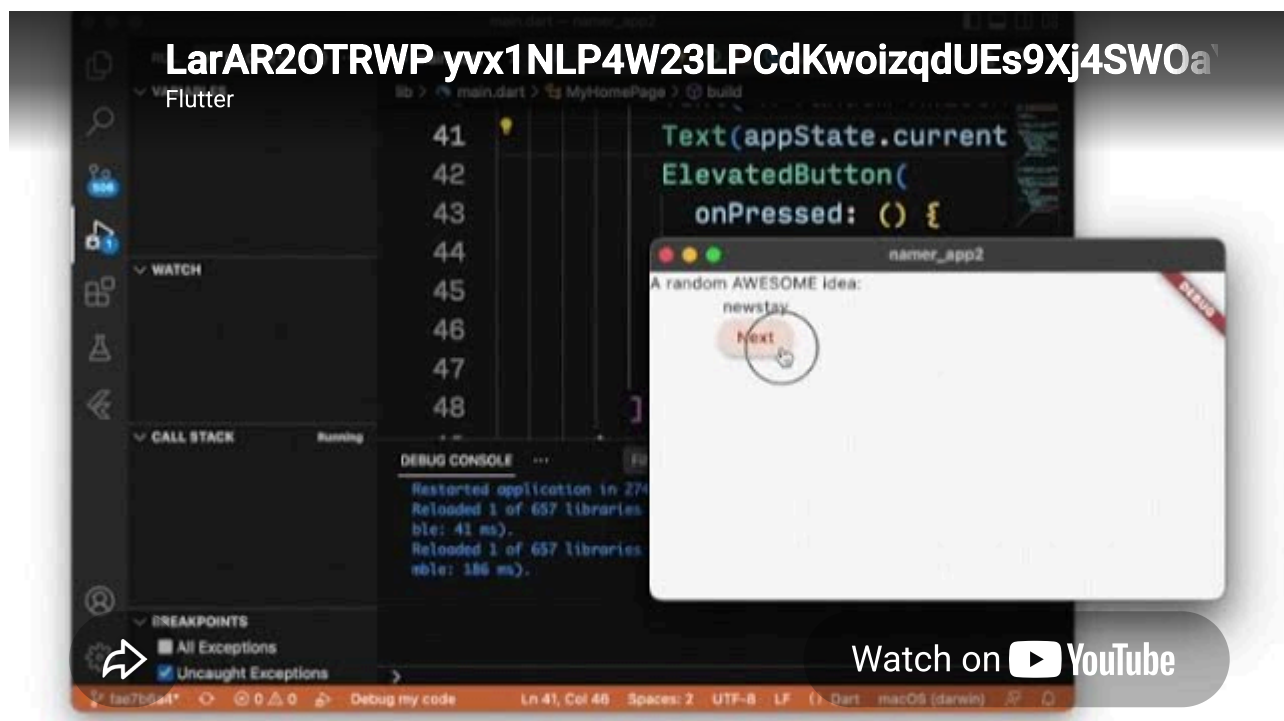
```
// ...

return Scaffold(
  body: Column(
    children: [
      Text('A random AWESOME idea:'),
      Text(appState.current.asLowerCase),

      // ↓ Add this.
      ElevatedButton(
        onPressed: () {
          print('button pressed!');
        },
        child: Text('Next'),
      ),
    ],
  ),
);

// ...
```

儲存變更後，應用程式會再次更新：畫面上會顯示按鈕，點選後，VS Code 的「Debug Console」會顯示「button pressed!」訊息。



5 分鐘 Flutter 速成課程

雖然觀看 偵錯控制台 很有趣，但您希望按鈕執行更有意義的操作。不過，在此之前，請先仔細查看 `lib/main.dart` 中的程式碼，瞭解其運作方式。

lib/main.dart

```
// ...

void main() {
  runApp(MyApp());
}

// ...
```

在檔案最上方，您會看到 `main()` 函式。目前這個檔案只會告知 Flutter 執行 `MyApp` 中定義的應用程式。

lib/main.dart

```
// ...

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return ChangeNotifierProvider(
      create: (context) => MyAppState(),
      child: MaterialApp(
        title: 'Namer App',
        theme: ThemeData(
          colorScheme: ColorScheme.fromSeed(seedColor: Colors.deepOrange),
        ),
        home: MyHomePage(),
      ),
    );
  }
}

// ...
```

`MyApp` 類別會擴充 `StatelessWidget`。小工具是建構每個 Flutter 應用程式的元素。如您所見，連應用程式本身也是小工具。

注意：稍後會說明 `StatelessWidget` (與 `StatefulWidget` 相比)。

`MyApp` 中的程式碼會設定整個應用程式。這段程式碼會建立應用程式範圍的狀態 (稍後會詳細說明)、命名應用程式、定義視覺主題，以及設定「首頁」小工具 (應用程式的起點)。

lib/main.dart

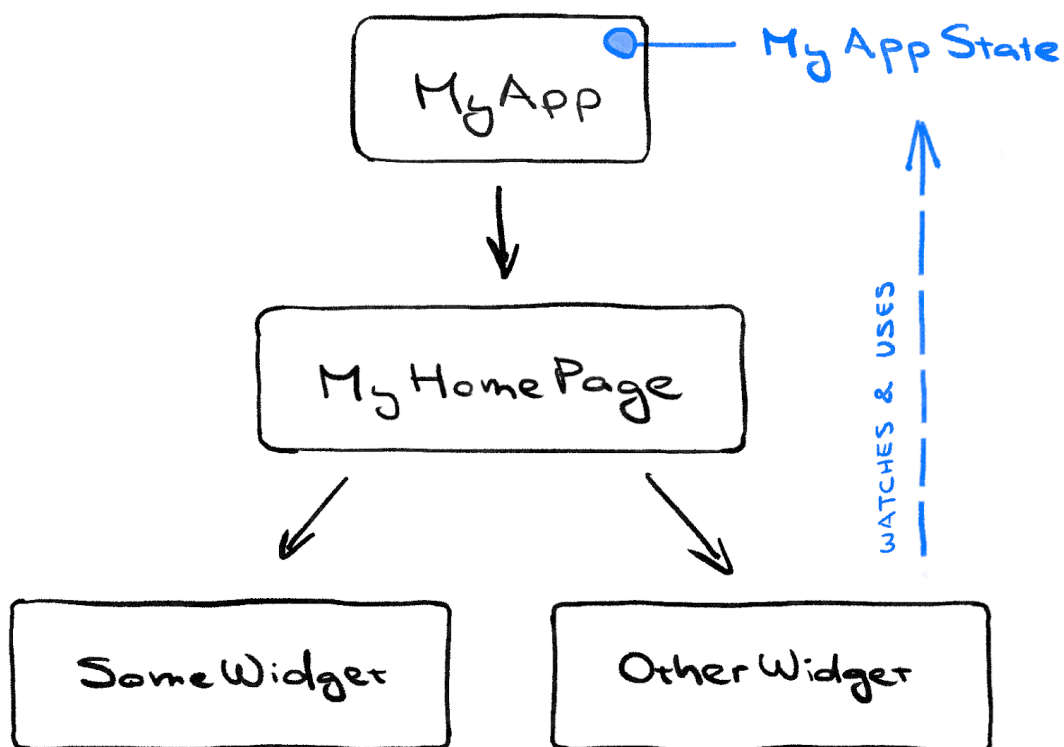
```
// ...

class MyAppState extends ChangeNotifier {
  var current = WordPair.random();
}

// ...
```

接著，`MyAppState` 類別會定義應用程式的狀態。這是您第一次接觸 Flutter，因此本程式碼研究室會盡量簡化內容，並著重於重點。在 Flutter 中，管理應用程式狀態的方法有很多種，最容易說明的是 `ChangeNotifier`，也就是這個應用程式採用的方法。

- `MyAppState` 定義應用程式運作所需的資料。目前只包含一個變數，也就是目前的隨機字詞配對。稍後會再新增內容。
- 狀態類別會擴充 `ChangeNotifier`，因此可以通知其他項目自身變更。舉例來說，如果目前的字詞配對有所變更，應用程式中的某些小工具就需要知道。
- 系統會使用 `ChangeNotifierProvider` 建立狀態並提供給整個應用程式 (請參閱上方 `MyApp` 中的程式碼)。這樣一來，應用程式中的任何小工具都能取得狀態。



lib/main.dart

```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) { // ← 1
    var appState = context.watch<MyAppState>(); // ← 2

    return Scaffold( // ← 3
      body: Column( // ← 4
        children: [
          Text('A random AWESOME idea:'), // ← 5
          Text(appState.current.asLowerCase), // ← 6
          ElevatedButton(
            onPressed: () {
              print('button pressed!');
            },
            child: Text('Next'),
          ),
        ], // ← 7
      ),
    );
  }
}

// ...
```

最後是 `MyHomePage`，也就是您已修改的小工具。下方的每個編號行都會對應至上方程式碼中的行號註解：

1. 每個小工具都會定義 `build()` 方法，每當小工具的狀況發生變化時，系統就會自動呼叫這個方法，確保小工具一律為最新狀態。
2. `MyHomePage` 會使用 `watch` 方法追蹤應用程式目前狀態的變更。
3. 每個 `build` 方法都必須傳回小工具，或 (更常見) 傳回巢狀小工具樹狀結構。在本例中，頂層小工具為 `Scaffold`。在本程式碼研究室中，您不會使用 `Scaffold`，但這是實用的小工具，在絕大多數的實際 Flutter 應用程式中都會用到。
4. `Column` 是 Flutter 最基本的版面配置小工具之一。可接受任意數量的子項，並從上到下放入欄中。根據預設，資料欄會在頂端放置子項。您很快就會變更此設定，讓資料欄置中對齊。
5. 您在第一個步驟中變更了這個 `Text` 小工具。
6. 這個第二個 `Text` 小工具會採用 `appState`，並存取該類別的唯一成員 `current` (這是 `WordPair`)。 `WordPair` 提供多個實用的擷取器，例如 `asPascalCase` 或 `asSnakeCase`。我們在此使用 `asLowerCase`，但如果您偏好其他替代方案，現在可以變更。
7. 請注意，Flutter 程式碼大量使用尾隨逗號。這個逗號不需要放在這裡，因為 `children` 是這個 `Column` 參數清單的最後一個 (也是唯一) 成員。不過，一般來說，使用尾隨逗號是個好主意：這樣可以輕鬆新增更多成員，而且還能提示 Dart 的自動格式化工具在此處換行。詳情請參閱「[程式碼格式化](#)」。

接著，請將按鈕連結至狀態。

您的第一個行為

捲動至 `MyAppState`，然後新增 `getNext` 方法。

lib/main.dart

```
// ...

class MyAppState extends ChangeNotifier {
  var current = WordPair.random();

  // ↓ Add this.
  void getNext() {
    current = WordPair.random();
    notifyListeners();
  }
}

// ...
```

新的 `getNext()` 方法會使用新的隨機 `WordPair` 重新指派 `current`。此外，它也會呼叫 `notifyListeners()` (`ChangeNotifier`) 的方法，可確保所有觀看 `MyAppState` 的使用者都會收到通知。

剩下的工作是從按鈕的回呼呼叫 `getNext` 方法。

lib/main.dart

```
// ...

ElevatedButton(
  onPressed: () {
    appState.getNext(); // ← This instead of print().
  },
  child: Text('Next'),
),

// ...
```

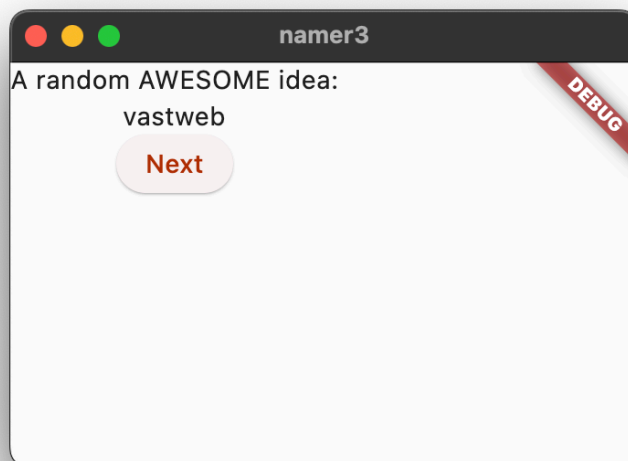
儲存並立即試用應用程式。每次按下「下一步」按鈕時，系統應會產生新的隨機字詞配對。

在下一節中，您將美化使用者介面。

步驟 5 · 約 15 分鐘

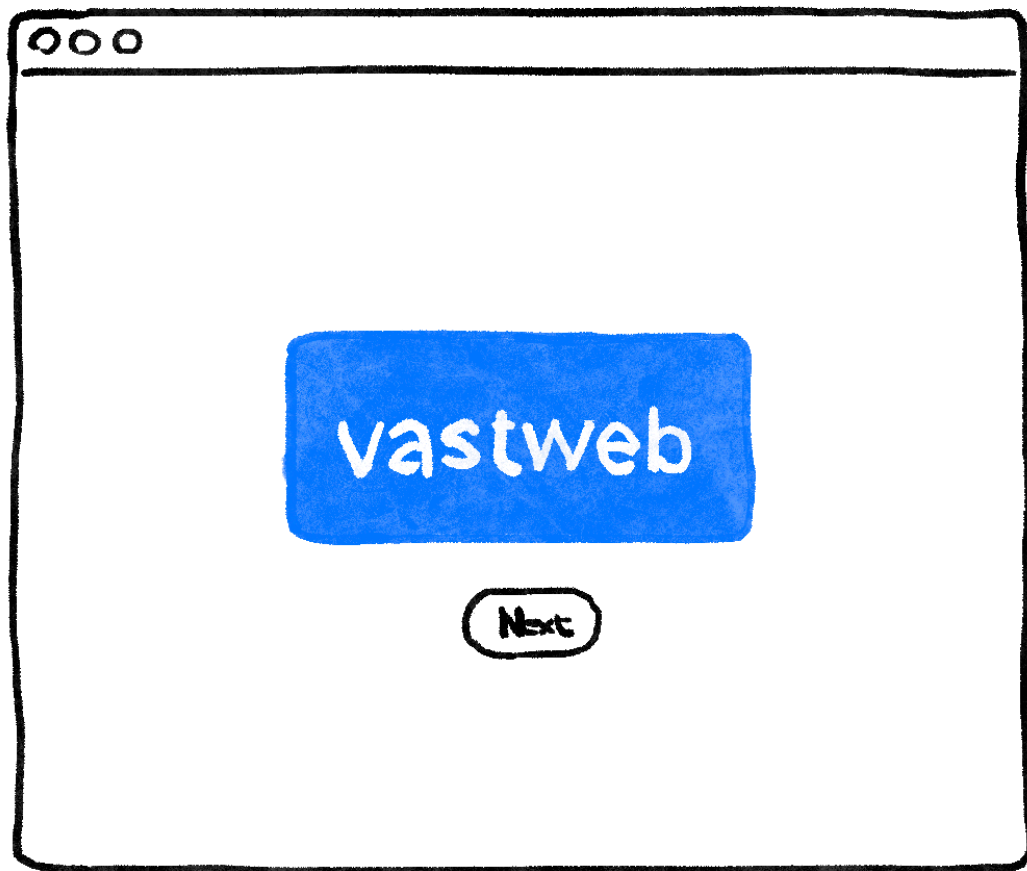
5. 讓應用程式更美觀

應用程式目前如下所示。



不佳。應用程式的核心內容 (隨機產生的字詞配對) 應更顯眼。畢竟，這才是使用者使用這款應用程式的主要原因！此外，應用程式內容會奇怪地偏離中心，而且整個應用程式都是無趣的黑白畫面。

本節將著重於應用程式設計，解決這些問題。本節的最終目標如下：



擷取小工具

負責顯示目前字詞配對的程式碼行現在看起來像這樣：`Text(appState.current.asLowerCase)`。如要將其變更為更複雜的內容，建議將這行程式碼擷取至個別小工具。為 UI 的不同邏輯部分提供個別的小工具，是管理 Flutter 複雜性的重要方式。

Flutter 提供重構輔助工具，可協助您擷取小工具，但使用前請務必確認擷取的程式碼行只會存取所需內容。目前，該行會存取 `appState`，但實際上只需要知道目前的字詞配對。

因此，請按照下列方式重新編寫 `MyHomePage` 小工具：

[lib/main.dart](#)

```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current; // ← Add this.

    return Scaffold(
      body: Column(
        children: [
          Text('A random AWESOME idea:'),
          Text(pair.asLowerCase), // ← Change to this.
          ElevatedButton(
            onPressed: () {
              appState.getNext();
            },
            child: Text('Next'),
          ),
        ],
      ),
    );
  }
}

// ...
```

真棒！`Text` 小工具不再參照整個 `appState`。

現在，請叫出「重構」選單。在 VS Code 中，您可以透過下列兩種方式執行這項操作：

1. 在要重構的程式碼片段 (本例為 `Text`) 上按一下滑鼠右鍵，然後從下拉式選單中選取「重構...」，

或

2. 將游標移至要重構的程式碼片段 (本例為 `Text`)，然後按下 `Ctrl+.` (Win/Linux) 或 `Cmd+.` (Mac)。



在「重構」選單中，選取「擷取小工具」。指派名稱，例如 **BigCard**，然後按一下 **Enter**。

這會在目前檔案的結尾自動建立新的類別 `BigCard`。該類別如下所示：

lib/main.dart

```
// ...

class BigCard extends StatelessWidget {
  const BigCard({super.key, required this.pair});

  final WordPair pair;

  @override
  Widget build(BuildContext context) {
    return Text(pair.asLowerCase);
  }
}

// ...
```

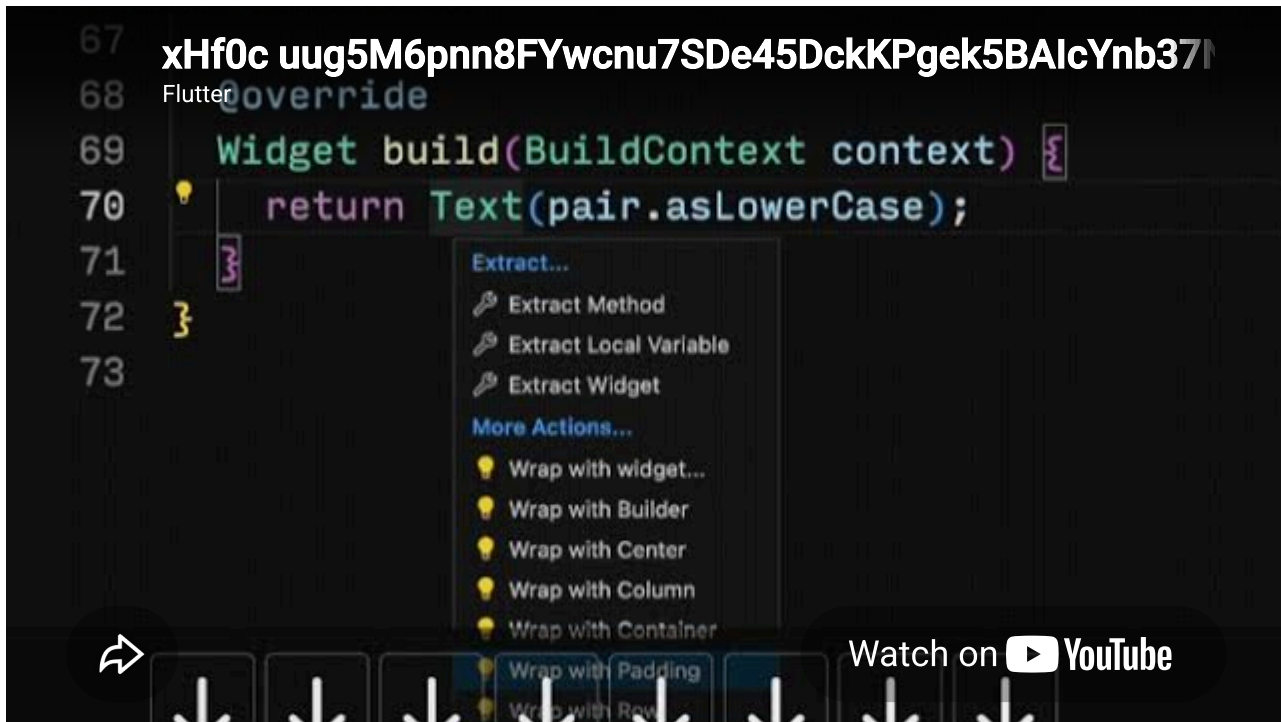
請注意，即使經過重構，應用程式仍會繼續運作。

新增信用卡

現在，請將這個新小工具打成本節開頭所設想的粗體 UI 片段。

找出 `BigCard` 類別和其中的 `build()` 方法。如先前所述，在 `Text` 小工具上叫出「重構」選單。不過，這次您不會擷取小工具。

請改為選取「Wrap with Padding」（使用邊框間距換行）。這項操作會在 `Text` 小工具周圍建立名為 `Padding` 的新父項小工具。儲存後，你會發現隨機字詞的空間變大了。



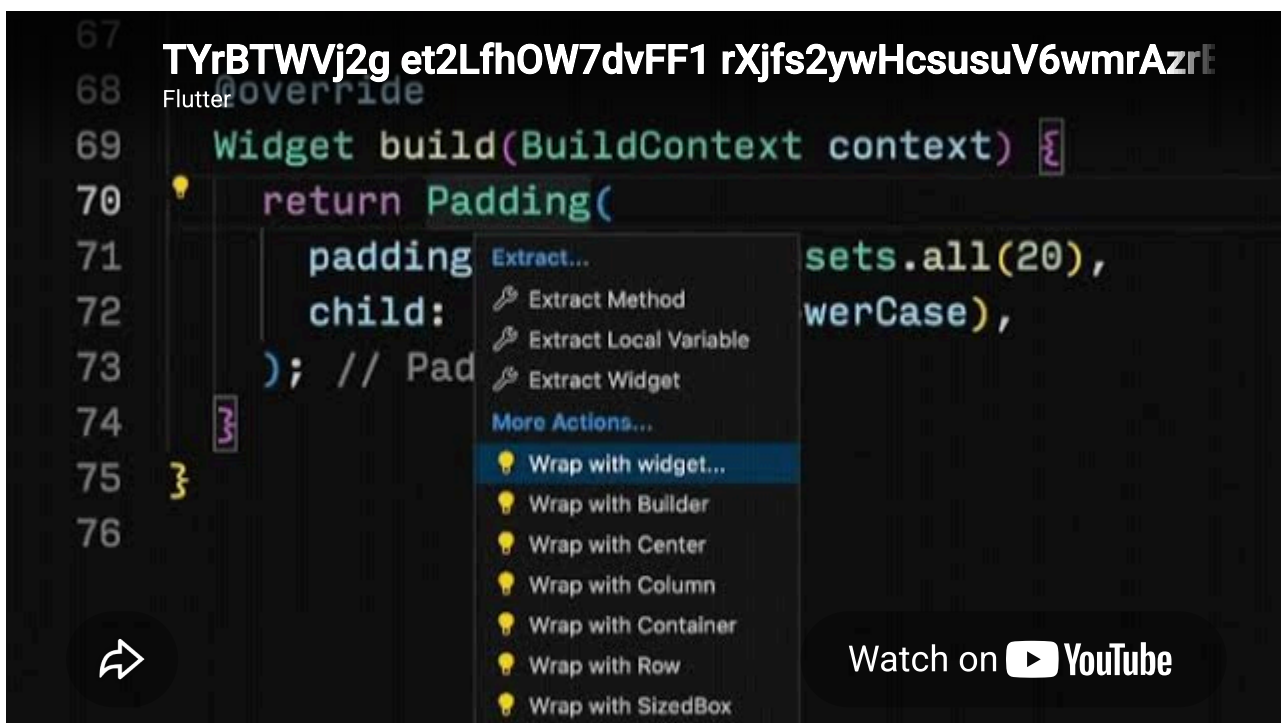
將邊框間距從預設值 8.0 增加。舉例來說，如要使用較寬鬆的邊框間距，請使用 20。

注意：Flutter 會盡可能使用組合而非繼承。在這裡，邊框間距不是 `Text` 的屬性，而是小工具！

這樣一來，小工具就能專注於單一職責，而開發人員可以完全自由地組合 UI。舉例來說，您可以使用 `Padding` 小工具填補文字、圖片、按鈕、自訂小工具或整個應用程式。小工具不會在意要包裝的內容。

接著返回上一層。將游標放在 `Padding` 小工具上，拉出「重構」選單，然後選取「Wrap with widget...」。

這樣您就能指定父項小工具。輸入「Card」並按下 **Enter** 鍵。



這會使用 `Card` 小工具包裝 `Padding` 小工具，因此也會包裝 `Text`。

lib/main.dart

```
// ...

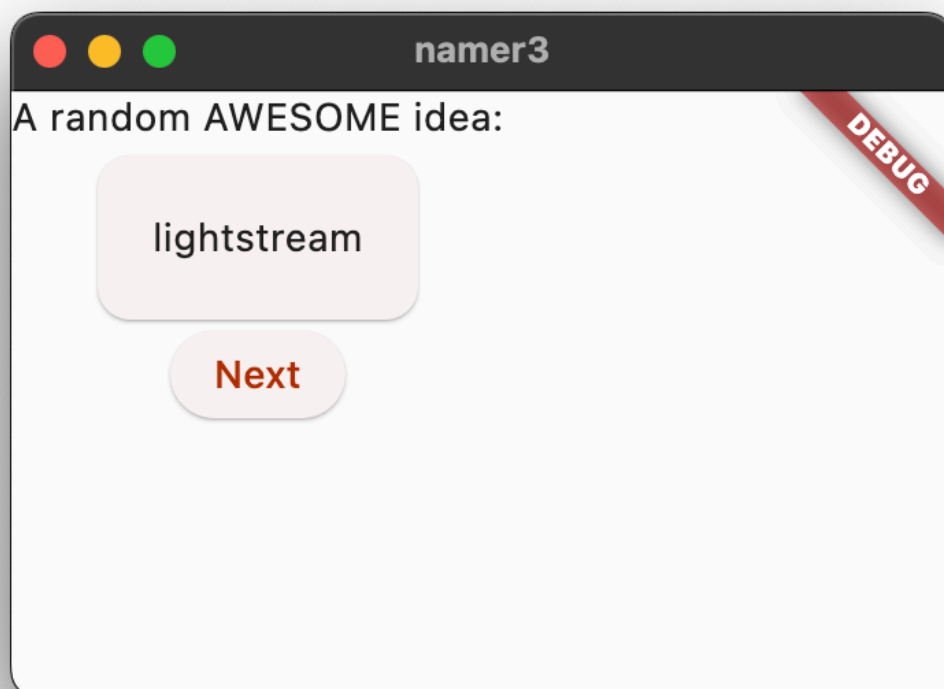
class BigCard extends StatelessWidget {
  const BigCard({super.key, required this.pair});

  final WordPair pair;

  @override
  Widget build(BuildContext context) {
    return Card(
      child: Padding(
        padding: const EdgeInsets.all(20),
        child: Text(pair.asLowerCase),
      ),
    );
  }
}

// ...
```

應用程式現在應如下所示：



主題和樣式

如要讓卡片更顯眼，請使用更豐富的色彩。此外，為了維持一致的色彩配置，請使用應用程式的 `Theme` 選擇顏色。

對 `BigCard` 的 `build()` 方法進行下列變更。

lib/main.dart

```
// ...

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);          // ← Add this.

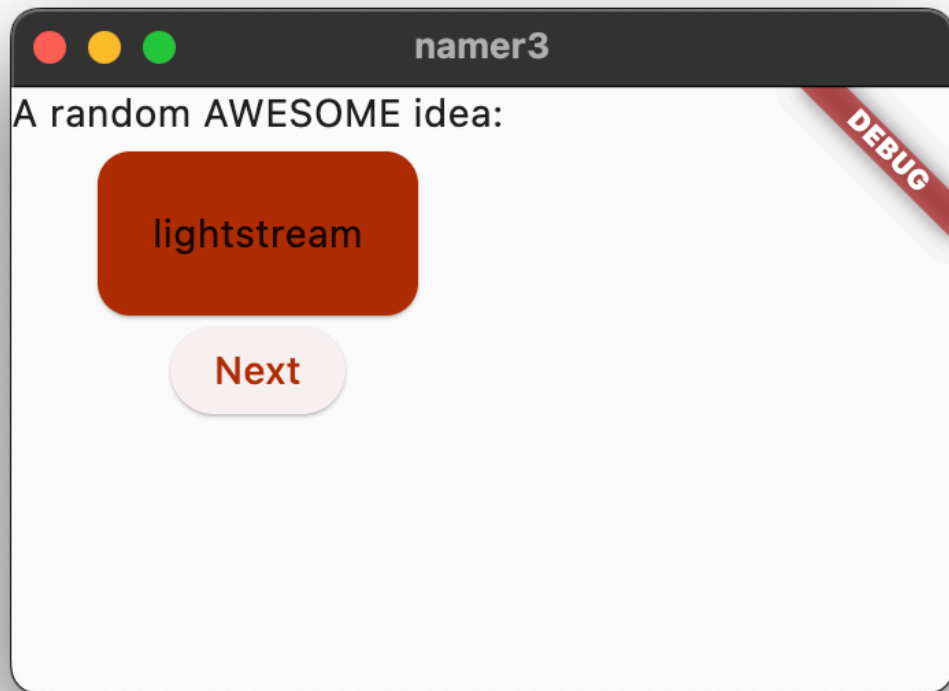
  return Card(
    color: theme.colorScheme.primary,      // ← And also this.
    child: Padding(
      padding: const EdgeInsets.all(20),
      child: Text(pair.asLowerCase),
    ),
  );
}

// ...
```

這兩行新程式碼會執行許多工作：

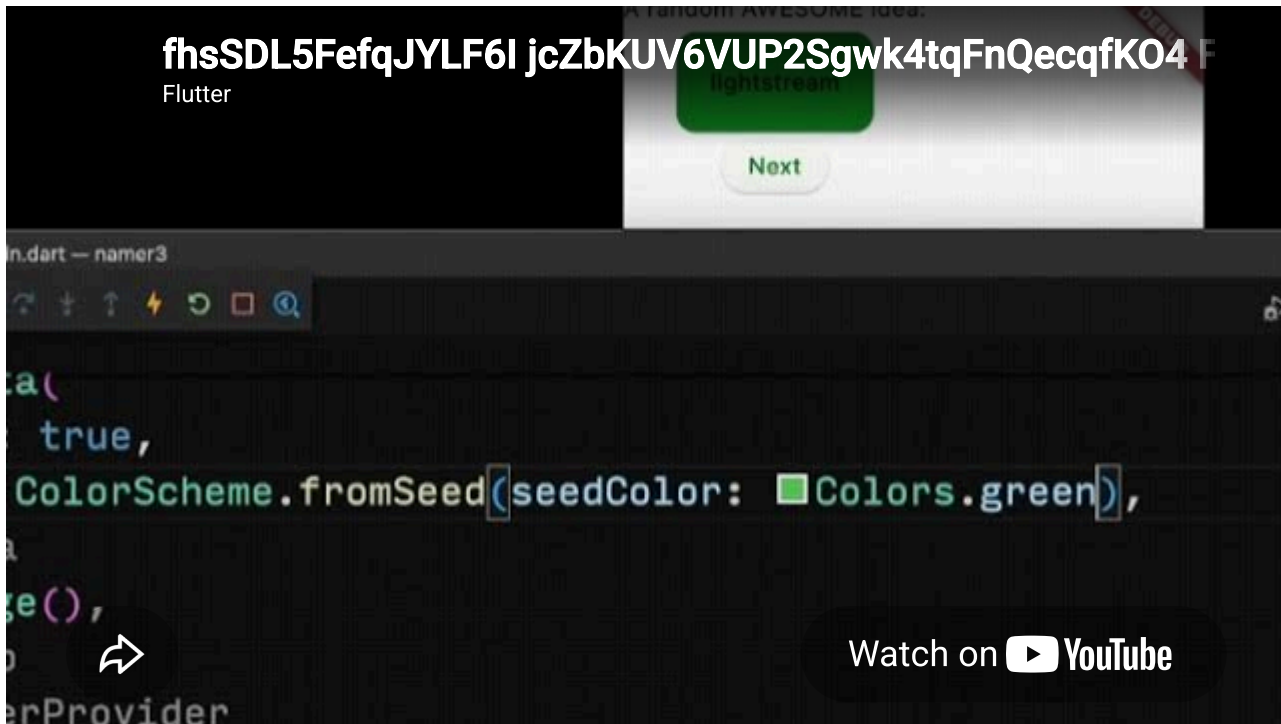
- 首先，程式碼會使用 `Theme.of(context)` 要求應用程式的目前主題。
- 接著，程式碼會將卡片的顏色定義為與主題的 `colorScheme` 屬性相同。色彩配置包含許多顏色，而 `primary` 是最顯眼的顏色，定義了應用程式的顏色。

現在，應用程式的主要顏色會套用至卡片：



如要變更這個顏色和整個應用程式的色彩配置，請向上捲動至 `MyApp`，然後在 `ColorScheme` 中變更種子顏色。

提示：Flutter 的 `Colors` 類別可讓您輕鬆存取精心挑選的調色盤，例如 `Colors.deepOrange` 或 `Colors.red`。當然，你也可以選擇任何顏色。舉例來說，如要定義完全不透明的純綠色，請使用 `Color.fromRGBO(0, 255, 0, 1.0)`。如果您喜歡十六進位數字，隨時可以選擇 `Color(0xFF00FF00)`。



請注意色彩如何順暢地產生動畫效果。這稱為「隱含動畫」。許多 Flutter 小工具都會在值之間平滑地插補，讓 UI 不會在狀態之間「跳躍」。

卡片下方的凸顯按鈕也會變更顏色。這就是使用應用程式範圍 `Theme` 的優點，而非硬式編碼值。

TextTheme

卡片仍有問題：文字太小，且顏色難以閱讀。如要修正這個問題，請對 `BigCard` 的 `build()` 方法進行下列變更。

lib/main.dart

```
// ...

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  // ↓ Add this.
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );

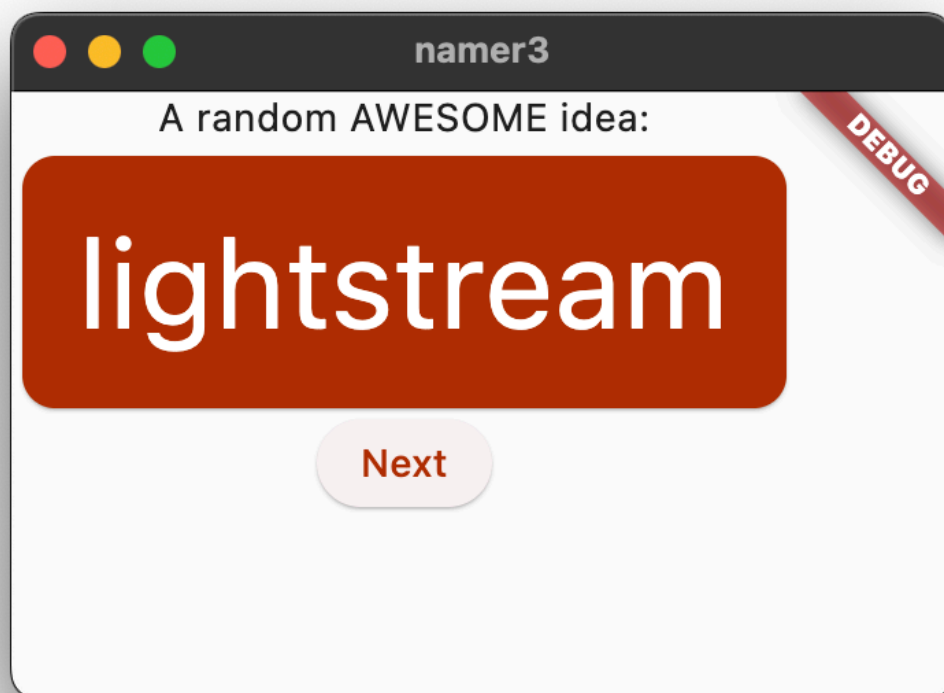
  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),
      // ↓ Change this line.
      child: Text(pair.asLowerCase, style: style),
    ),
  );
}

// ...
```

這項異動的原因：

- 使用 `theme.textTheme`，即可存取應用程式的字型主題。這個類別包含 `bodyMedium` (適用於中等大小的標準文字)、`caption` (適用於圖片說明) 或 `headlineLarge` (適用於大型標題) 等成員。
- `displayMedium` 屬性是適用於顯示文字的大型樣式。這裡的「顯示」一詞是排版用語，例如「顯示字體」。 `displayMedium` 的說明文件指出「顯示樣式會保留給簡短且具重要性的文字」，這正是我們的用途。
- 理論上，主題的 `displayMedium` 屬性可以是 `null`。您用來編寫這個應用程式的程式設計語言 Dart 具有空值安全性，因此不會讓您呼叫可能為 `null` 的物件方法。不過，在這種情況下，您可以使用 `!` 運算子 (「bang 運算子」) 向 Dart 保證您知道自己在做什麼。(在這種情況下，`displayMedium` 絕對不是 `null`。(我們知道這點的原因不在本程式碼研究室的範圍內)。
- 在 `displayMedium` 上呼叫 `copyWith()` 會傳回文字樣式的副本，並套用您定義的變更。在本例中，您只會變更文字顏色。
- 如要取得新顏色，請再次存取應用程式的主題。色彩配置的 `onPrimary` 屬性定義的顏色，很適合用於應用程式的主要顏色上。

應用程式現在看起來應該會像這樣：



如果願意，可以進一步變更卡片。不妨參考下列建議：

- `copyWith()` 可讓你變更文字樣式的許多屬性，而不只是顏色。如要取得可變更的完整屬性清單，請將游標放在 `copyWith()` 的括號內，然後按下 `Ctrl+Shift+Space` (Windows/Linux) 或 `Cmd+Shift+Space` (Mac)。
- 同樣地，你也可以變更 `Card` 小工具的更多設定。舉例來說，您可以增加 `elevation` 參數的值，放大卡片的陰影。
- 試著使用不同的顏色。除了 `theme.colorScheme.primary` 以外，還有 `.secondary`、`.surface` 和其他無數的旗標。這些顏色都有對應的 `onPrimary`。

提升無障礙體驗

Flutter 預設會讓應用程式可供存取。舉例來說，每個 Flutter 應用程式都會正確地向 TalkBack 和 VoiceOver 等螢幕閱讀器顯示應用程式中的所有文字和互動式元素。



但有時需要進行一些作業。以這個應用程式為例，螢幕閱讀器可能無法正確發音某些產生的字詞配對。人類可以輕鬆辨識「cheaphead」中的兩個字，但螢幕閱讀器可能會將字中間的「ph」發音為「f」。

解決方法是將 `pair.asLowerCase` 替換為 `"${pair.first} ${pair.second}"`。後者會使用字串插補，從 `pair` 中包含的兩個字詞建立字串 (例如 "cheap head")。使用兩個獨立字詞而非複合字，可確保螢幕閱讀器正確識別，並為視障使用者提供更優質的體驗。

不過，您可能希望維持 `pair.asLowerCase` 的視覺簡潔性。使用 `Text` 的 `semanticsLabel` 屬性，以更適合螢幕閱讀器的語意內容，覆寫文字小工具的視覺內容：

[lib/main.dart](#)

```
// ...

@override
Widget build(BuildContext context) {
  final theme = Theme.of(context);
  final style = theme.textTheme.displayMedium!.copyWith(
    color: theme.colorScheme.onPrimary,
  );

  return Card(
    color: theme.colorScheme.primary,
    child: Padding(
      padding: const EdgeInsets.all(20),

      // ↓ Make the following change.
      child: Text(
        pair.asLowerCase,
        style: style,
        semanticsLabel: "${pair.first} ${pair.second}",
      ),
    ),
  );
}

// ...
```

現在，螢幕閱讀器會正確發音每個生成的字詞配對，但 UI 保持不變。如要實際體驗這項功能，請[在裝置上使用螢幕閱讀器](#)。

提示：Flutter 提供多種無障礙工具，包括自動測試和 `Semantics` 小工具。如要瞭解詳情，請參閱 Flutter 說明文件的[無障礙功能頁面](#)。

將 UI 置中

現在隨機字詞配對已呈現足夠的視覺風格，接下來請將其放在應用程式視窗/畫面的中央。

首先，請注意 `BigCard` 是 `Column` 的一部分。根據預設，欄會將子項集中在頂端，但我們可以覆寫這項設定。前往 `MyHomePage` 的 `build()` 方法，然後進行下列變更：

[lib/main.dart](#)

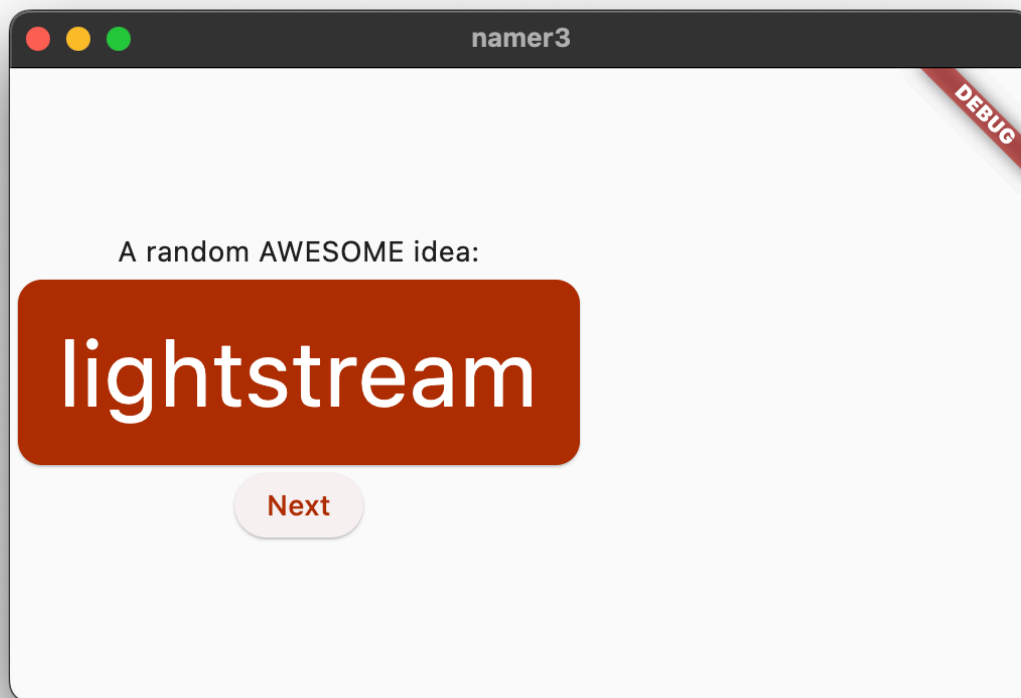
```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

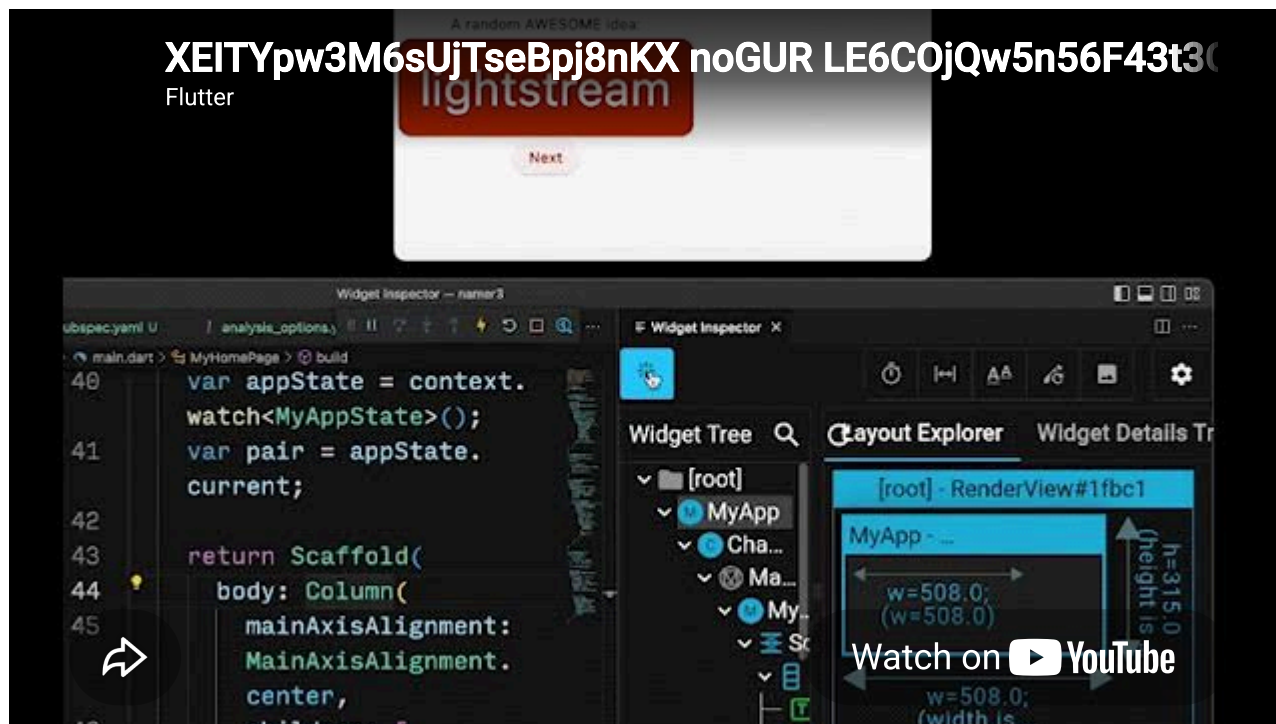
    return Scaffold(
      body: Column(
        mainAxisAlignment: MainAxisAlignment.center, // ← Add this.
        children: [
          Text('A random AWESOME idea:'),
          BigCard(pair: pair),
          ElevatedButton(
            onPressed: () {
              appState.getNext();
            },
            child: Text('Next'),
          ),
        ],
      ),
    );
  }
}

// ...
```

這會讓 `Column` 內的子項沿著主要 (垂直) 軸置中。

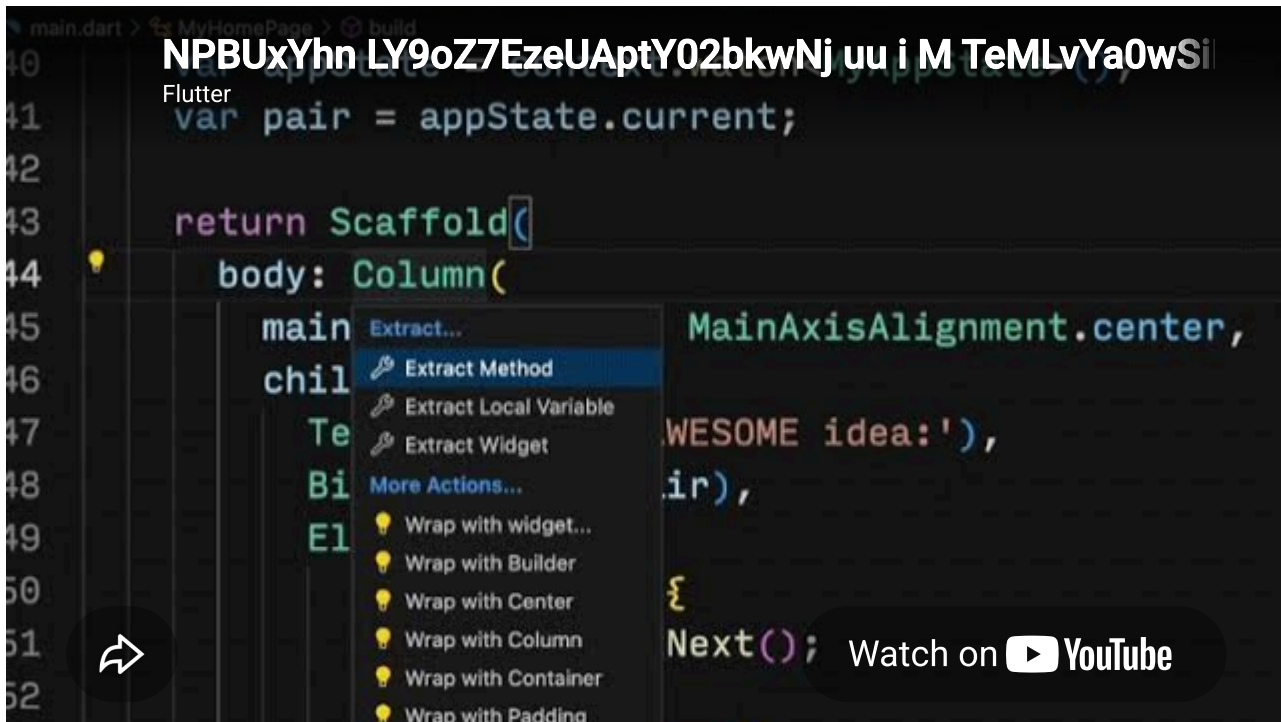


子項已沿著欄的交叉軸置中 (也就是已水平置中)。但 `Column` 本身並未置中於 `Scaffold` 內。我們可以使用小工具檢查器驗證這點。

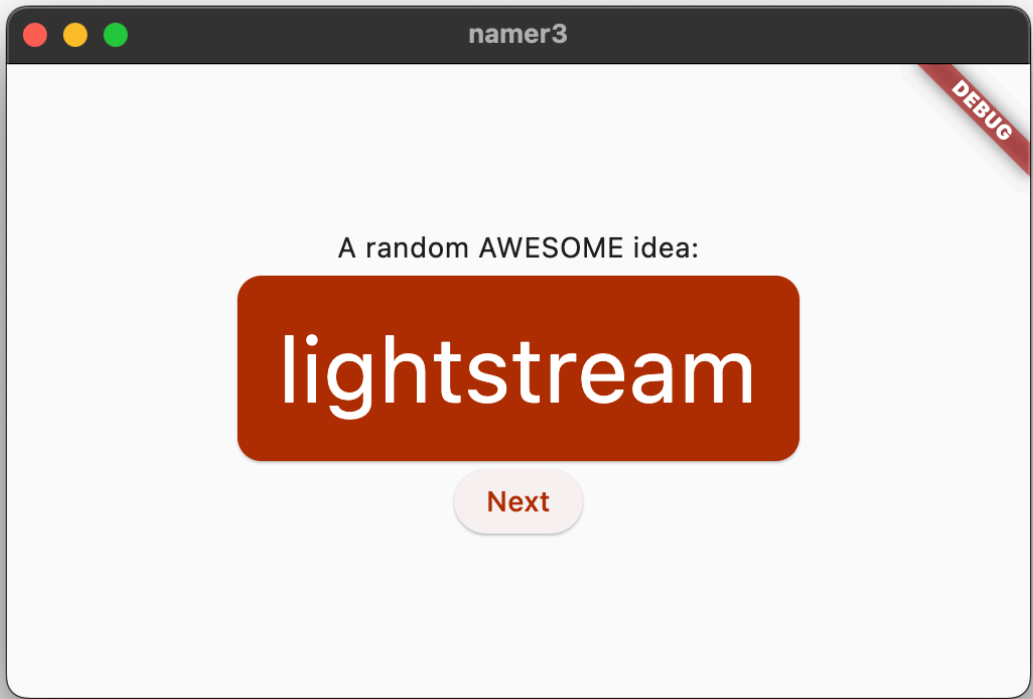


本程式碼研究室不會深入探討 Widget Inspector，但您可以發現，當 `Column` 遭到醒目顯示時，並不會佔用應用程式的整個寬度，只會佔用子項所需的水平空間。

您只需要將資料欄置中即可。將游標放在 `Column` 上，叫出「重構」選單 (使用 `Ctrl+. 或 Cmd+. ），然後選取「Wrap with Center」。`



應用程式現在看起來應該會像這樣：



如有需要，可以進一步調整。

- 你可以移除「Text」小工具上方的「BigCard」。可以說，即使沒有描述性文字 (「一個隨機的超棒點子：」)，使用者也能理解 UI，因此不再需要這段文字。這樣也比較乾淨。
- 您也可以在 BigCard 和 ElevatedButton 之間新增 SizedBox(height: 10) 小工具。這樣一來，兩個小工具之間就會有更多間隔。SizedBox 小工具只會佔用空間，本身不會算繪任何內容。通常用來建立視覺「間隙」。

選用變更後，`MyHomePage` 會包含下列程式碼：

`lib/main.dart`

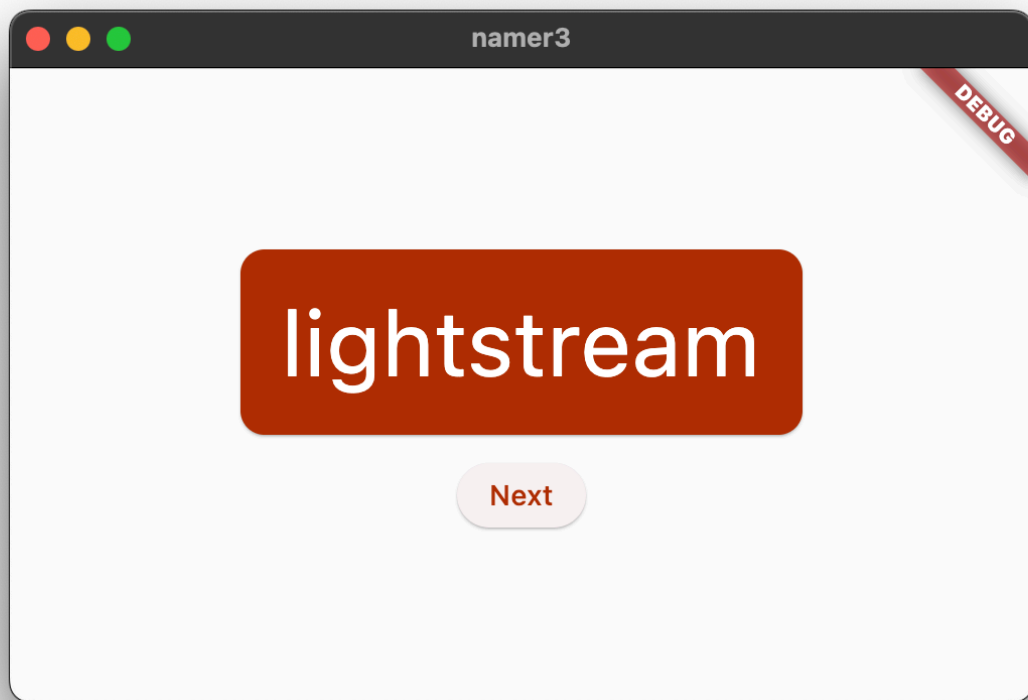
```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    return Scaffold(
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            BigCard(pair: pair),
            SizedBox(height: 10),
            ElevatedButton(
              onPressed: () {
                appState.getNext();
              },
              child: Text('Next'),
            ),
          ],
        ),
      ),
    );
  }
}

// ...
```

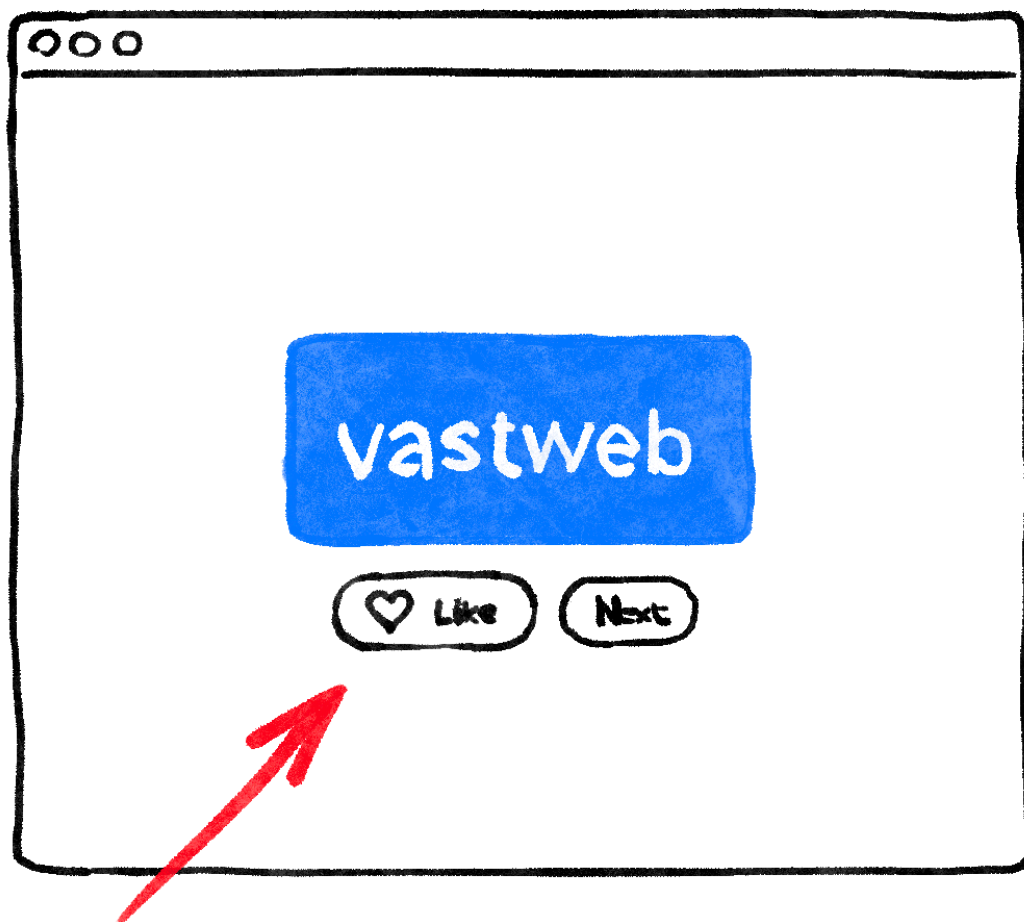
應用程式如下所示：



在下一節中，您將新增將生成字詞加入「我的最愛」(或「喜歡」) 的功能。

6. 新增功能

這個應用程式可以正常運作，有時甚至會提供有趣的字詞配對。但使用者每次點選「下一步」，就會永久消失一組字詞。最好能「記住」最佳建議，例如提供「喜歡」按鈕。



新增業務邏輯

捲動至 `MyAppState`，然後加入下列程式碼：

[lib/main.dart](#)

```
// ...

class MyAppState extends ChangeNotifier {
  var current = WordPair.random();

  void getNext() {
    current = WordPair.random();
    notifyListeners();
  }

  // ↓ Add the code below.
  var favorites = <WordPair>[];

  void toggleFavorite() {
    if (favorites.contains(current)) {
      favorites.remove(current);
    } else {
      favorites.add(current);
    }
    notifyListeners();
  }
}

// ...
```

查看變更：

- 您已在 `MyAppState` 中新增名為 `favorites` 的屬性。這個屬性會以空白清單初始化：`[]`。
- 您也指定清單只能包含字詞配對：`<WordPair>[]`，使用[泛型](#)。這有助於提升應用程式的穩定性，因為如果您嘗試在其中加入 `WordPair` 以外的任何內容，Dart 甚至會拒絕執行應用程式。因此，您可以放心使用 `favorites` 清單，因為其中絕不會有任何不必要的物件 (例如 `null`)。

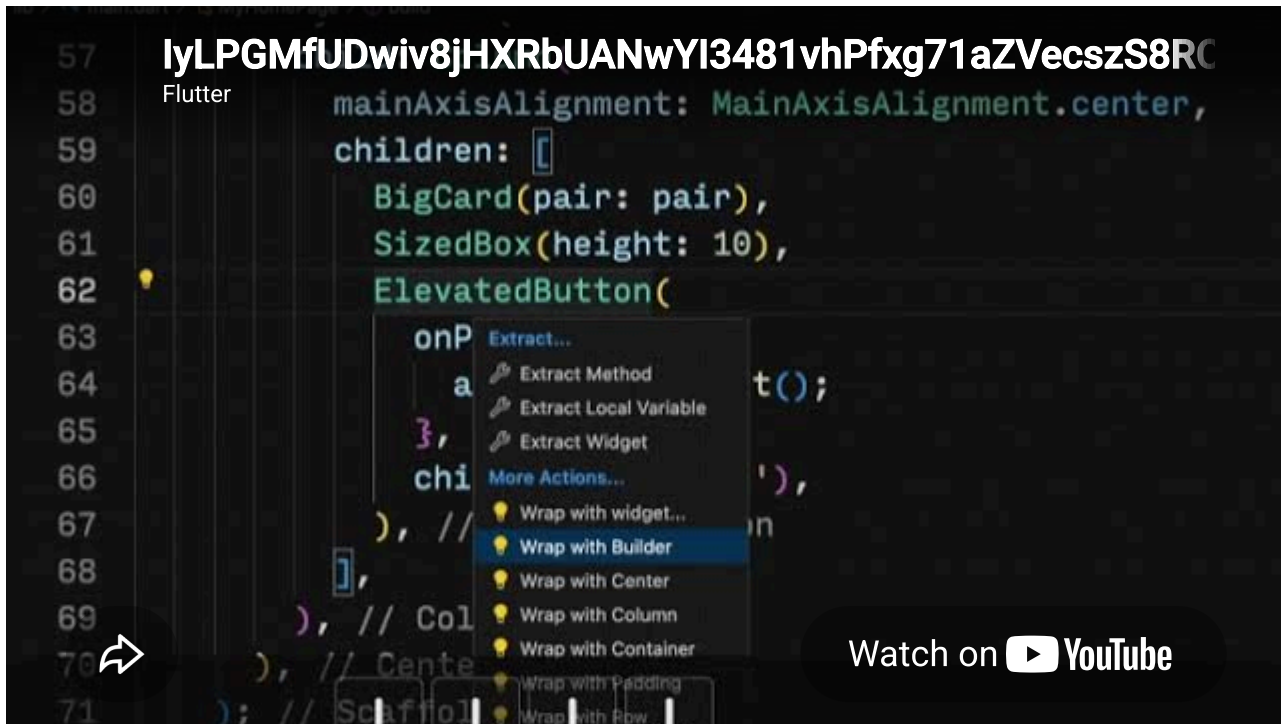
注意：Dart 除了 `List` (以 `[]` 表示) 之外，還有其他集合型別。您可能會認為以 `Set` (以 `{}` 表示) 建立收藏項目集合會更有意義。為了盡可能簡化本程式碼研究室，我們將使用清單。但如要使用 `Set`，程式碼不會有太大變化。

- 您也新增了 `toggleFavorite()` 方法，可從我的最愛清單中移除目前的字詞配對 (如果已在清單中)，或新增字詞配對 (如果尚未加入)。無論是哪種情況，程式碼都會在之後呼叫 `notifyListeners();`。

新增按鈕

解決「商業邏輯」問題後，接下來要再次處理使用者介面。如要將「喜歡」按鈕放在「繼續」按鈕左側，必須使用 `Row`。 `Row` 小工具是 `Column` 的水平對應項目，您先前已看過。

首先，將現有按鈕包裝在 `Row` 中。前往 `MyHomePage` 的 `build()` 方法，將游標放在 `ElevatedButton` 上，使用 `Ctrl+.` 或 `Cmd+.` 呼叫「重構」選單，然後選取「Wrap with Row」。



儲存後，您會發現 `Row` 的行為與 `Column` 類似，預設會將子項集中在左側。(`Column` 將子項集中在頂端)。如要修正這個問題，可以採用與先前相同的方法，但要使用 `mainAxisAlignment`。不過，基於教學 (學習) 目的，請使用 `mainAxisSize`。這會告知 `Row` 不要佔用所有可用的水平空間。

進行下列變更：

[lib/main.dart](#)

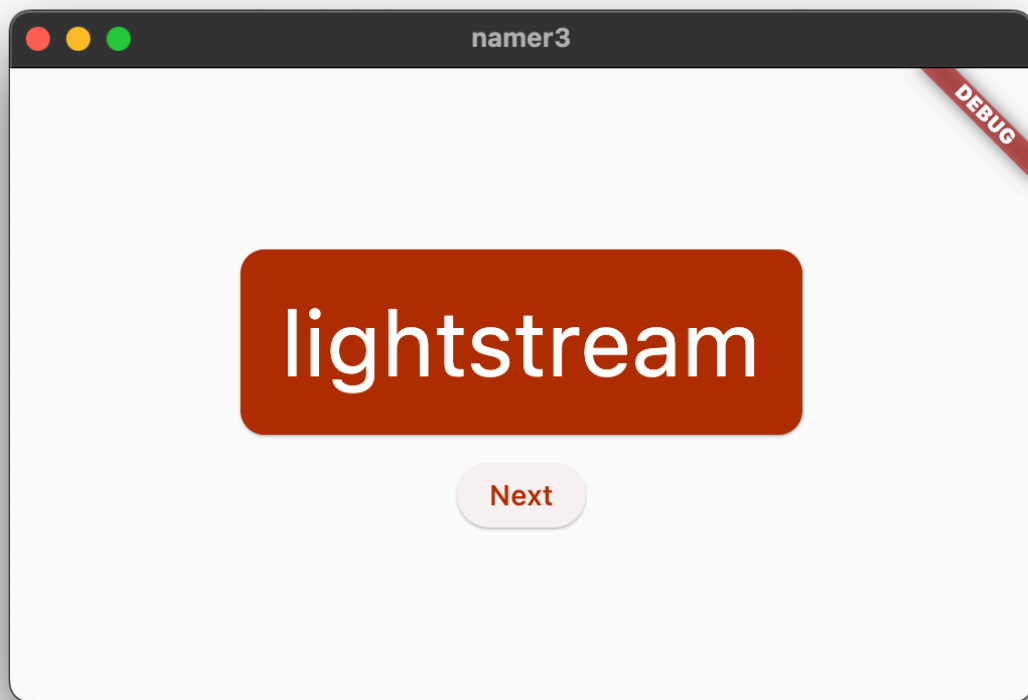
```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

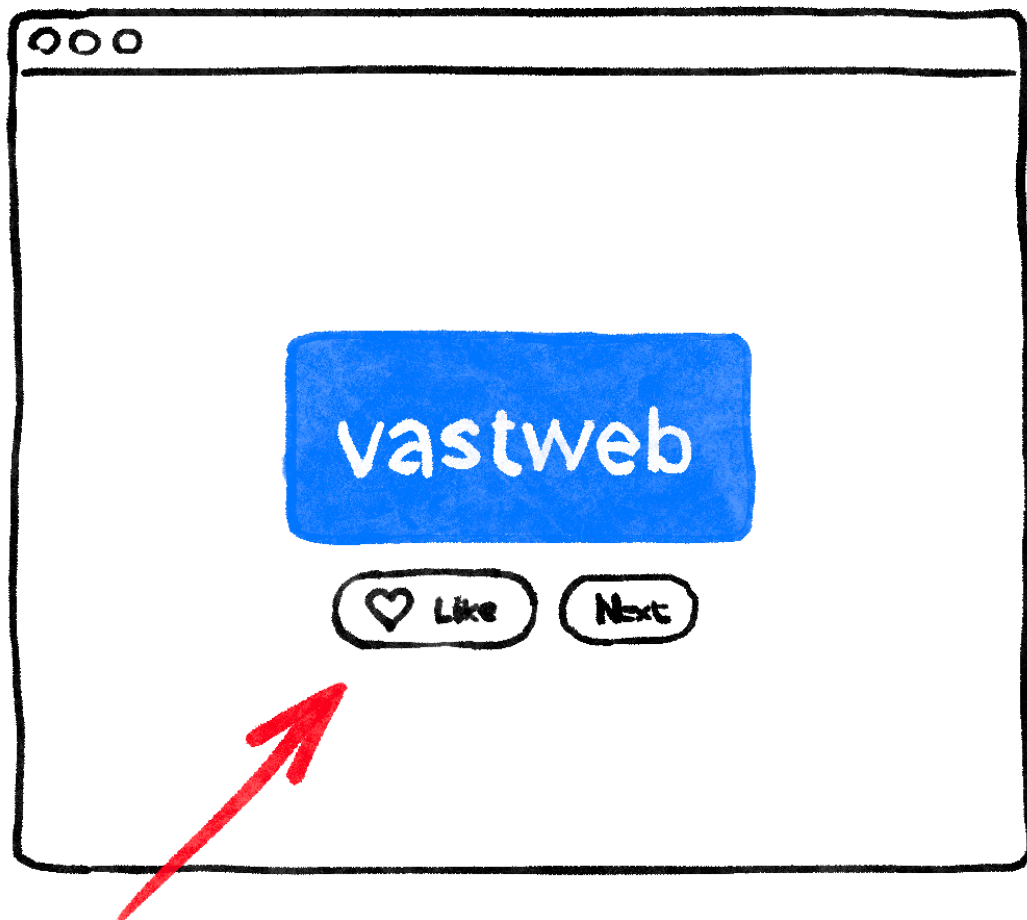
    return Scaffold(
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            BigCard(pair: pair),
            SizedBox(height: 10),
            Row(
              mainAxisAlignment: MainAxisAlignment.min, // ← Add this.
              children: [
                ElevatedButton(
                  onPressed: () {
                    appState.getNext();
                  },
                  child: Text('Next'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}

// ...
```

使用者介面會恢復原狀。



接著，新增「喜歡」按鈕，並連結至 `toggleFavorite()`。建議您先自行完成這項挑戰，不要查看下方的程式碼區塊。



即使做法與下文不盡相同，也沒關係。事實上，除非你真的想接受重大挑戰，否則不必擔心愛心圖示。

即使失敗也沒關係，畢竟這是你第一次接觸 Flutter。

↓ SPOILERS AHEAD ↓

以下是在 `MyHomePage` 中新增第二個按鈕的方法。這次請使用 `ElevatedButton.icon()` 建構函式建立含有圖示的按鈕。然後在 `build` 方法頂端，根據目前的字詞配對是否已加入我的最愛，選擇適當的圖示。此外，請注意再次使用 `SizeBox`，讓兩個按鈕保持一點距離。

[lib/main.dart](#)

```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    // ↓ Add this.
    IconData icon;
    if (appState.favorites.contains(pair)) {
      icon = Icons.favorite;
    } else {
      icon = Icons.favorite_border;
    }

    return Scaffold(
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            BigCard(pair: pair),
            SizedBox(height: 10),
            Row(
              mainAxisAlignment: MainAxisAlignment.min,
              children: [

                // ↓ And this.
                ElevatedButton.icon(
                  onPressed: () {
                    appState.toggleFavorite();
                  },
                  icon: Icon(icon),
                  label: Text('Like'),
                ),
                SizedBox(width: 10),

                ElevatedButton(
                  onPressed: () {
                    appState.getNext();
                  },
                  child: Text('Next'),
                ),
              ],
            ),
          ],
        ),
      ),
    );
  }
}
```


// ...

應用程式應如下所示：



很抱歉，使用者無法查看我的最愛。現在要為應用程式新增一整個獨立畫面。我們會在下一節中繼續說明！

步驟 7 · 約 20 分鐘

7. 新增導覽邊欄

大多數應用程式無法在單一畫面上顯示所有內容。這個應用程式可能可以，但為了教學目的，您要為使用者的最愛項目建立個別畫面。如要在這兩個畫面之間切換，您將實作第一個 `StatefulWidget`。



為了盡快進入這個步驟的重點，請將 `MyHomePage` 分成 2 個不同的小工具。

選取所有 `MyHomePage`，刪除並替換為下列程式碼：

[lib/main.dart](#)

```
// ...

class MyHomePage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [
                NavigationRailDestination(
                  icon: Icon(Icons.home),
                  label: Text('Home'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.favorite),
                  label: Text('Favorites'),
                ),
              ],
              selectedIndex: 0,
              onDestinationSelected: (value) {
                print('selected: $value');
              },
            ),
          ),
          Expanded(
            child: Container(
              color: Theme.of(context).colorScheme.primaryContainer,
              child: GeneratorPage(),
            ),
          ),
        ],
      ),
    );
  }
}

class GeneratorPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();
    var pair = appState.current;

    IconData icon;
    if (appState.favorites.contains(pair)) {
      icon = Icons.favorite;
    } else {
      icon = Icons.favorite_border;
    }
  }
}
```

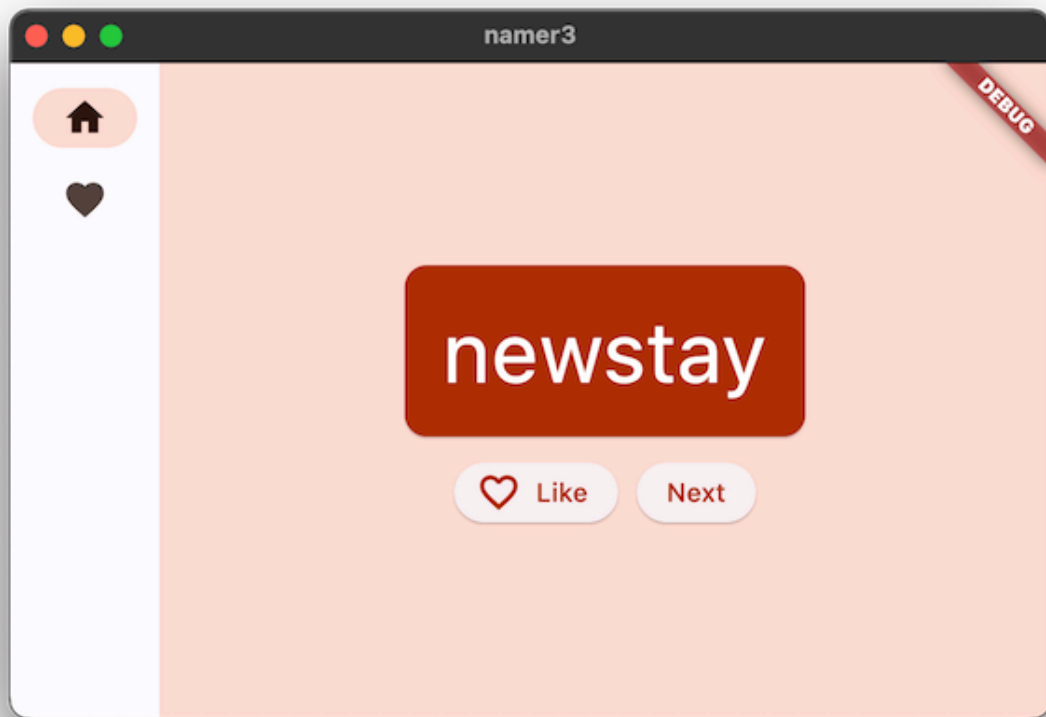
```

return Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      BigCard(pair: pair),
      SizedBox(height: 10),
      Row(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          ElevatedButton.icon(
            onPressed: () {
              appState.toggleFavorite();
            },
            icon: Icon(icon),
            label: Text('Like'),
          ),
          SizedBox(width: 10),
          ElevatedButton(
            onPressed: () {
              appState.getNext();
            },
            child: Text('Next'),
          ),
        ],
      ),
    ],
  ),
);
}
}

// ...

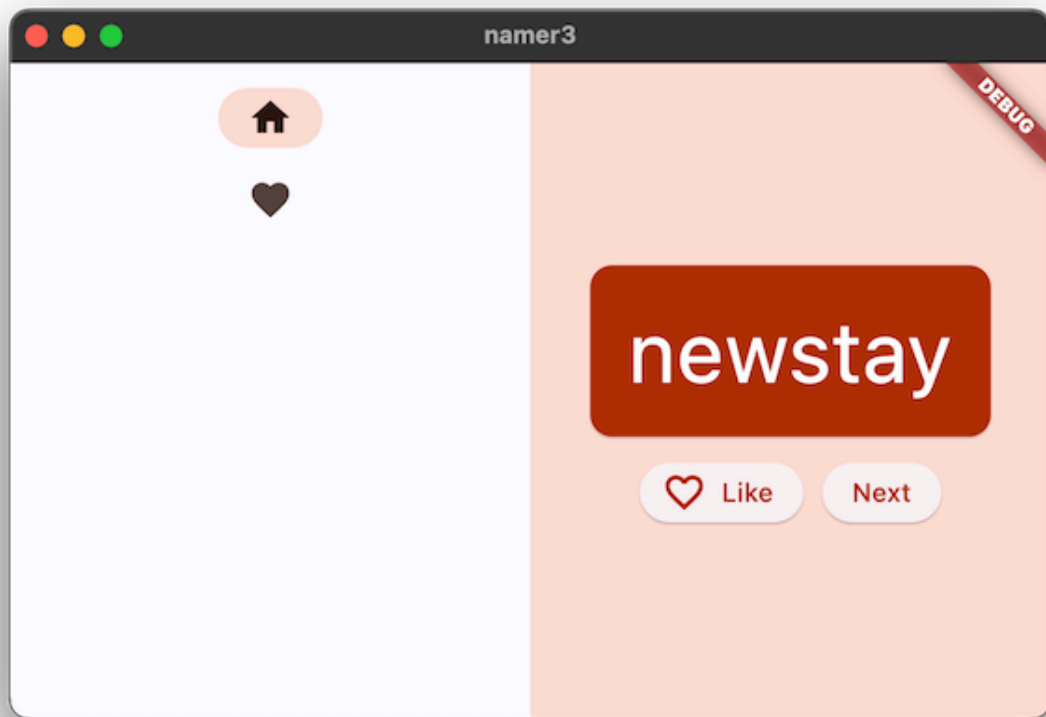
```

儲存後，您會發現 UI 的視覺化部分已準備就緒，但無法運作。按一下導覽列中的「愛心」♥ 不會執行任何動作。



檢查變更。

- 首先，請注意 `MyHomePage` 的所有內容都會擷取到新的小工具 `GeneratorPage` 中。舊版 `MyHomePage` 小工具中唯一未擷取的部分是 `Scaffold`。
- 新的 `MyHomePage` 包含兩個子項的 `Row`。第一個小工具是 `SafeArea`，第二個是 `Expanded` 小工具。
- `SafeArea` 可確保子項不會遭到硬體凹口或狀態列遮蔽。在這個應用程式中，小工具會包裝 `NavigationRail`，避免導覽按鈕遭到行動裝置狀態列遮蔽。
- 您可以在 `NavigationRail` 中將 `extended: false` 行變更為 `true`。圖示旁邊會顯示標籤。在後續步驟中，您將瞭解如何在應用程式有足夠的水平空間時，自動執行這項操作。
- 導覽軌有兩個目的地（「首頁」和「我的最愛」），以及各自的圖示和標籤。同時也會定義目前的 `selectedIndex`。選取的索引為零時，會選取第一個目的地；選取的索引為一時，會選取第二個目的地，依此類推。目前硬式編碼為零。
- 導覽軌也會定義使用者透過 `onDestinationSelected` 選取其中一個目的地時會發生的情況。目前，應用程式只會使用 `print()` 輸出所要求的索引值。
- `Row` 的第二個子項是 `Expanded` 小工具。擴展小工具在列和欄中非常實用，可讓您表示版面配置，其中部分子項只會佔用所需空間（在本例中為 `SafeArea`），其他小工具則應盡可能佔用剩餘空間（在本例中為 `Expanded`）。`Expanded` 小工具可視為「貪婪」的項目，如要進一步瞭解這個小工具的角色，請嘗試使用另一個 `Expanded` 包裝 `SafeArea` 小工具。產生的版面配置如下所示：



- 兩個 `Expanded` 小工具會將所有可用的水平空間一分为二，即使導覽側欄實際上只需要左側的一小部分空間。
- `Expanded` 小工具內有彩色 `Container`，容器內則有 `GeneratorPage`。

無狀態與有狀態的小工具

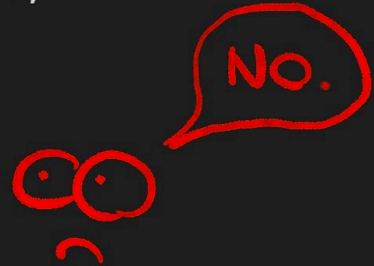
到目前為止，`MyAppState` 涵蓋了所有狀態需求。因此，您目前撰寫的所有小工具都是無狀態。不會包含任何可變動的狀態。任何小工具都無法自行變更，必須透過 `MyAppState` 變更。

但這種情況即將改變。

您需要某種方式來保留導覽側欄的 `selectedIndex` 值。您也希望能夠從 `onDestinationSelected` 回呼中變更這個值。

您可以將 `selectedIndex` 新增為 `MyAppState` 的另一個屬性。而且確實有效。但您可以想像，如果每個小工具都將值儲存在應用程式狀態中，應用程式狀態很快就會變得不合理地龐大。

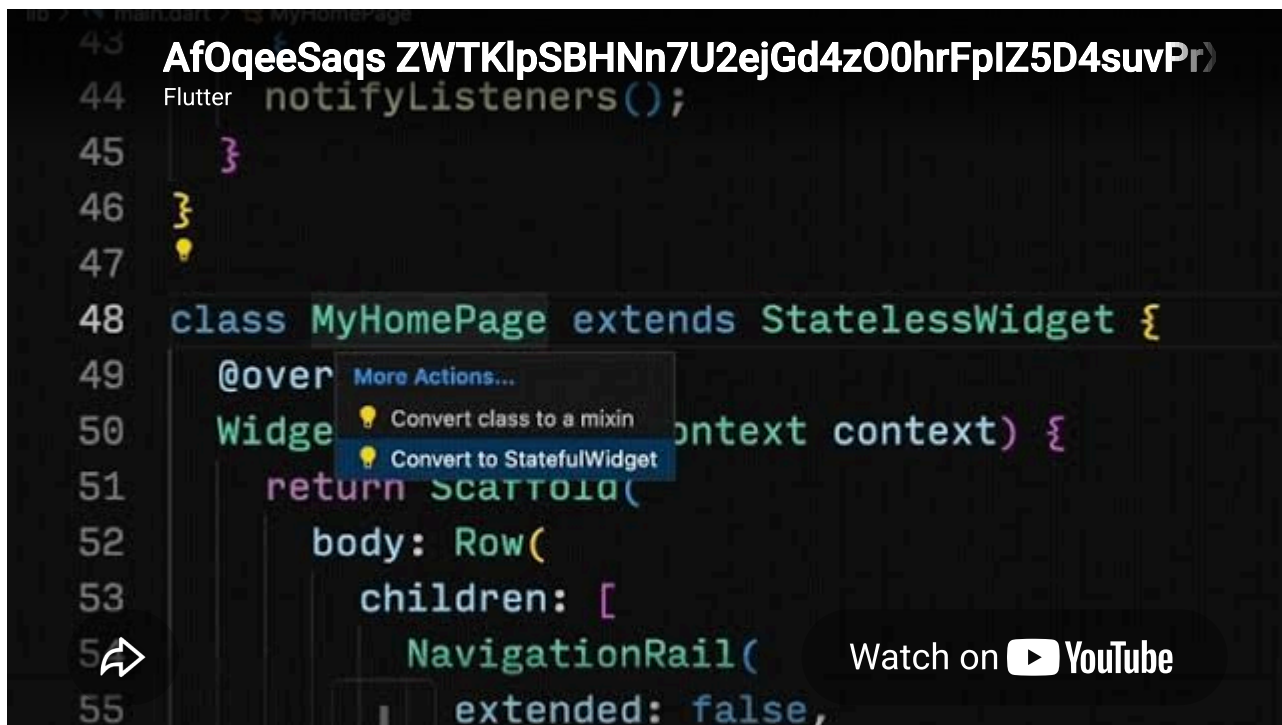
```
class MyAppState extends ChangeNotifier {
  var current = WordPair.random();
  var favorites = <WordPair>[];
  var selectedIndex = 0;
  var selectedIndexInAnotherWidget = 0;
  var indexInYetAnotherWidget = 42;
  var optionASelected = false;
  var optionBSelected = false;
  var loadingFromNetwork = false;
  // ...
}
```



有些狀態只與單一小工具相關，因此應保留在該小工具中。

輸入 `StatefulWidget`，這是一種含有 `State` 的小工具。首先，將 `MyHomePage` 轉換為有狀態的小工具。

將游標放在 `MyHomePage` 的第一行 (以 `class MyHomePage...` 開頭)，然後使用 `Ctrl+. 或 Cmd+. 呼叫「重構」選單。然後選取「Convert to StatefulWidget」(轉換為 StatefulWidget)。`



IDE 會為您建立新類別 `_MyHomePageState`。這個類別會擴充 `State`，因此可以管理自己的值。(這項設定本身可能會變更)。另請注意，舊無狀態小工具的 `build` 方法已移至 `_MyHomePageState` (而非留在小工具中)。這項作業會逐字移動，`build` 方法中的任何內容都不會變更。現在只是搬到其他地方。

開頭的底線 (`_`) 會將 `_MyHomePageState` 設為私有類別，並由編譯器強制執行。如要進一步瞭解 Dart 的隱私權和其他主題，請參閱[語言導覽](#)。

setState

新的有狀態小工具只需要追蹤一個變數：`selectedIndex`。對 `_MyHomePageState` 進行下列 3 項變更：

[lib/main.dart](#)


```
// ...

class _MyHomePageState extends State<MyHomePage> {

  var selectedIndex = 0;      // ← Add this property.

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [
                NavigationRailDestination(
                  icon: Icon(Icons.home),
                  label: Text('Home'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.favorite),
                  label: Text('Favorites'),
                ),
              ],
              selectedIndex: selectedIndex,    // ← Change to this.
              onDestinationSelected: (value) {

                // ↓ Replace print with this.
                setState(() {
                  selectedIndex = value;
                });

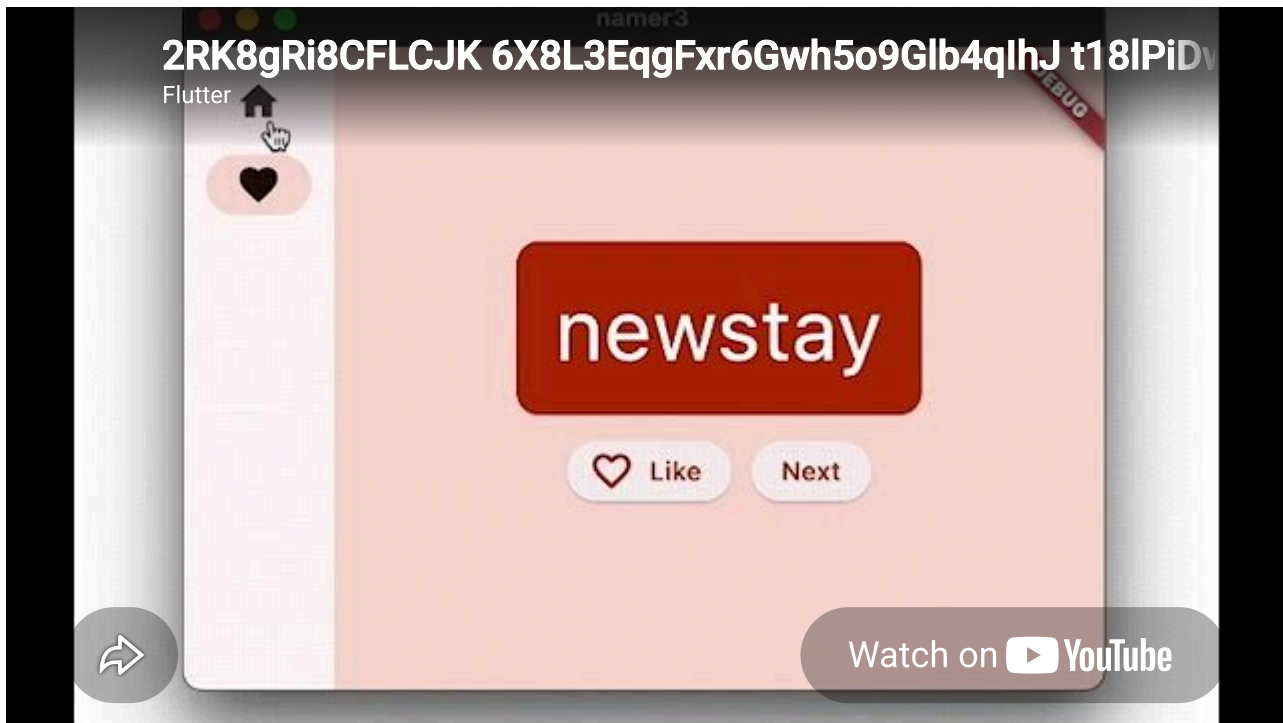
              },
            ),
          ),
          Expanded(
            child: Container(
              color: Theme.of(context).colorScheme.primaryContainer,
              child: GeneratorPage(),
            ),
          ),
        ],
      ),
    );
  }
}

// ...
```

查看變更：

1. 您會導入新變數 `selectedIndex`，並將其初始化為 `0`。

2. 您可以在 `NavigationRail` 定義中使用這個新變數，取代目前為止的硬式編碼 `0`。
3. 呼叫 `onDestinationSelected` 回呼時，您會在 `setState()` 呼叫中將新值指派給 `selectedIndex`，而不是僅將新值列印到控制台。這項呼叫與先前使用的 `notifyListeners()` 方法類似，可確保 UI 更新。



導覽軌現在會回應使用者互動。但右側的擴展區域維持不變。這是因為程式碼並未使用 `selectedIndex` 判斷要顯示哪個畫面。

使用 `selectedIndex`

將下列程式碼放在 `_MyHomePageState` 的 `build` 方法頂端，緊接在 `return Scaffold` 之前：

`lib/main.dart`

```
// ...

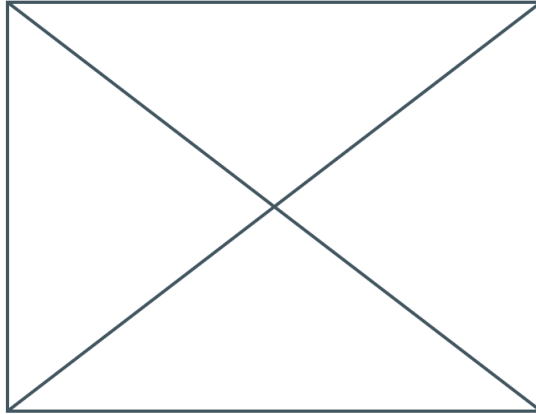
Widget page;
switch (selectedIndex) {
  case 0:
    page = GeneratorPage();
    break;
  case 1:
    page = Placeholder();
    break;
  default:
    throw UnimplementedError('no widget for $selectedIndex');
}

// ...
```

檢查這段程式碼：

1. 這段程式碼會宣告類型為 `Widget` 的新變數 `page`。

2. 接著，switch 陳述式會根據 `selectedIndex` 中的目前值，將畫面指派給 `page`。
3. 由於目前沒有 `FavoritesPage`，請使用 `Placeholder`。這個實用的小工具會在您放置的位置繪製交叉矩形，將該部分 UI 標示為未完成。



4. 套用「快速失敗原則」後，如果 `selectedIndex` 不是 0 或 1，switch 陳述式也會確保擲回錯誤。這有助於避免後續發生錯誤。如果您在導覽軌中新增目的地，但忘記更新這段程式碼，程式會在開發階段當機 (而不是讓您猜測為何無法運作，或讓您將有錯誤的程式碼發布到正式環境)。

現在 `page` 包含您要在右側顯示的小工具，您大概可以猜到還需要進行哪些變更。

以下是最後一項變更 `_MyHomePageState`：

[lib/main.dart](#)

```
// ...

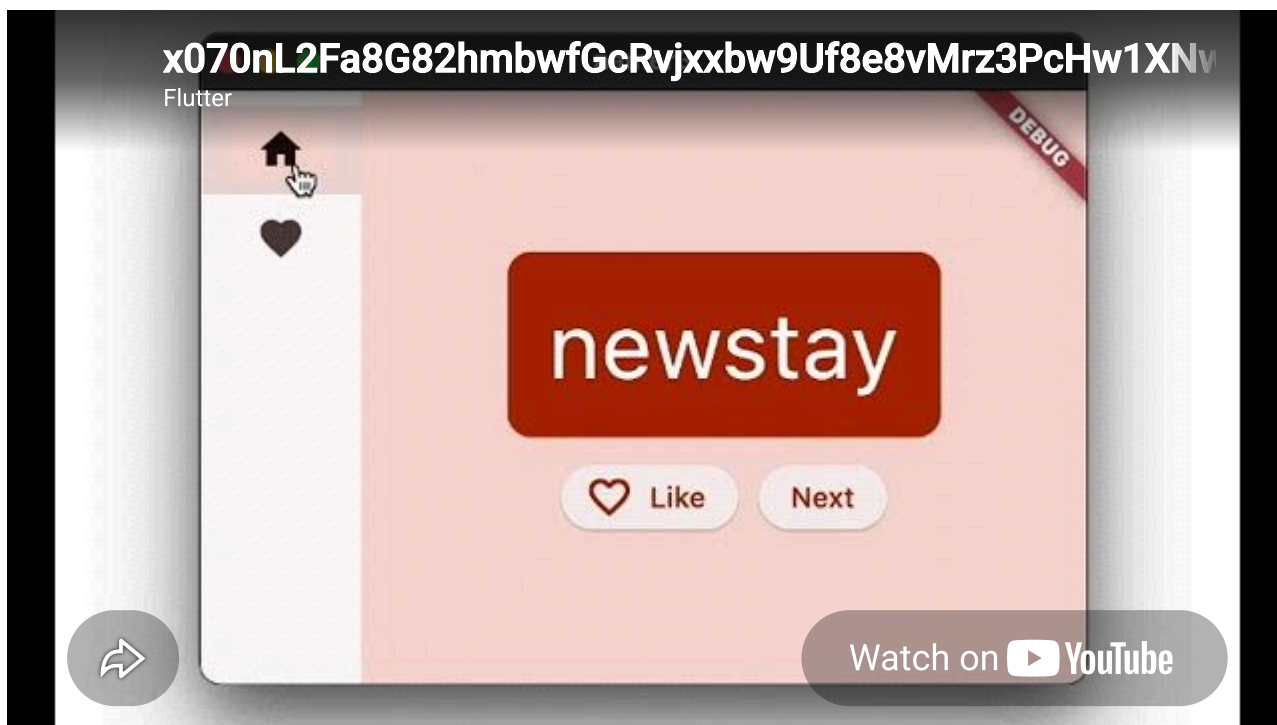
class _MyHomePageState extends State<MyHomePage> {
  var selectedIndex = 0;

  @override
  Widget build(BuildContext context) {
    Widget page;
    switch (selectedIndex) {
      case 0:
        page = GeneratorPage();
        break;
      case 1:
        page = Placeholder();
        break;
      default:
        throw UnimplementedError('no widget for $selectedIndex');
    }

    return Scaffold(
      body: Row(
        children: [
          SafeArea(
            child: NavigationRail(
              extended: false,
              destinations: [
                NavigationRailDestination(
                  icon: Icon(Icons.home),
                  label: Text('Home'),
                ),
                NavigationRailDestination(
                  icon: Icon(Icons.favorite),
                  label: Text('Favorites'),
                ),
              ],
              selectedIndex: selectedIndex,
              onDestinationSelected: (value) {
                setState(() {
                  selectedIndex = value;
                });
              },
            ),
          ),
          Expanded(
            child: Container(
              color: Theme.of(context).colorScheme.primaryContainer,
              child: page, // ← Here.
            ),
          ),
        ],
      ),
    );
  }
}
```

```
}  
}  
  
// ...
```

應用程式現在會在我們的 `GeneratorPage` 和預留位置之間切換，預留位置很快就會變成「我的最愛」頁面。



積極回應

接著，讓導覽邊欄具備回應式設計。也就是說，在有足夠空間時，使用 `extended: true` 自動顯示標籤。



Flutter 提供多種小工具，可協助您自動調整應用程式版面配置，舉例來說，`Wrap` 是類似於 `Row` 或 `Column` 的小工具，當垂直或水平空間不足時，會自動將子項換行 (稱為「執行」)。`FittedBox` 是一種小工具，可根據您的規格，自動將子項調整至可用空間。

但 `NavigationRail` 不會自動顯示標籤，因為系統無法判斷每個情境中「足夠的空間」是多少。您 (開發人員) 必須發出該呼叫。

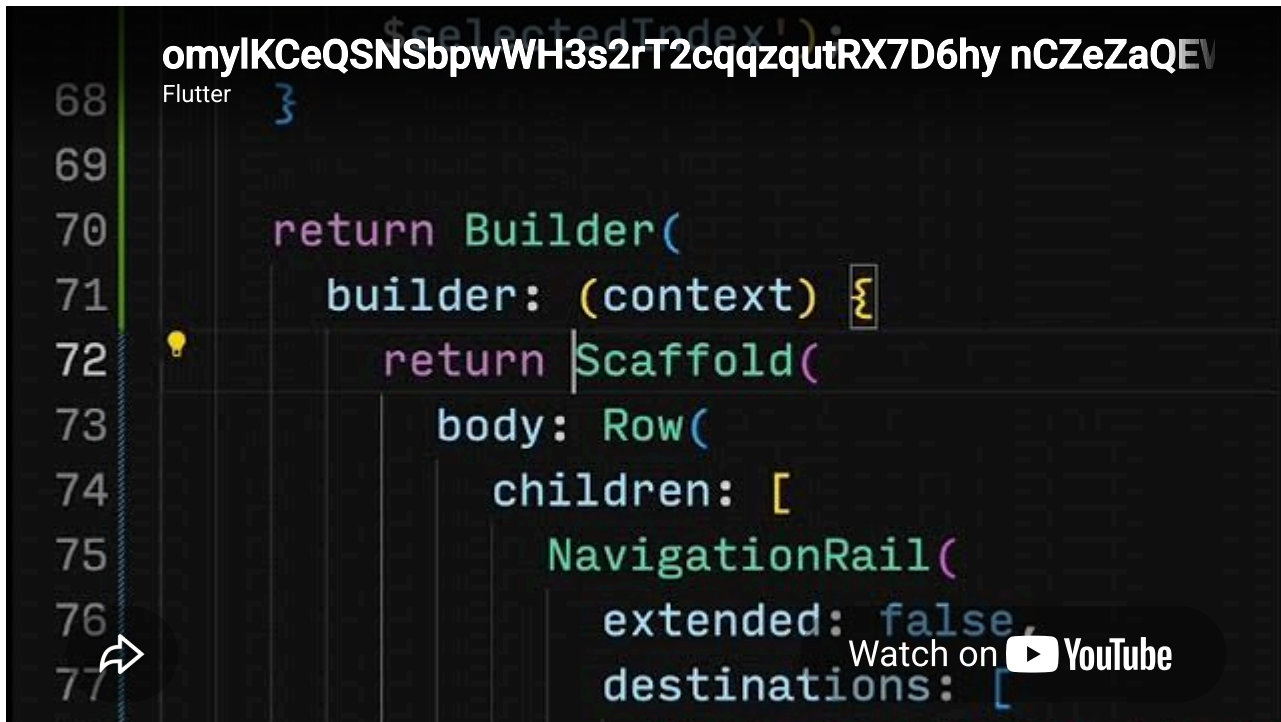
假設您決定只有在 `MyHomePage` 寬度至少為 600 像素時才顯示標籤。

注意：Flutter 會以邏輯像素做為長度單位。有時也稱為與裝置無關的像素。無論應用程式是在舊款低解析度手機上執行，還是在新版「視網膜」裝置上執行，8 像素的邊框間距在視覺上都是相同的。實體螢幕每公分約有 38 個邏輯像素，或每英吋約有 96 個邏輯像素。

在本例中，要使用的小工具是 `LayoutBuilder`。您可以根據可用空間大小變更小工具樹狀結構。

再次使用 VS Code 中的 Flutter 「重構」選單，進行必要變更。不過，這次的情況稍微複雜一點：

1. 在 `_MyHomePageState` 的 `_MyHomePageState` 方法中，將游標放在 `Scaffold` 上。`build`
2. 使用 `Ctrl+.` (Windows/Linux) 或 `Cmd+.` (Mac) 叫出「重構」選單。
3. 選取「Wrap with Builder」(使用 Builder 包裝)，然後按下 `Enter` 鍵。
4. 將新增的 `Builder` 名稱修改為 `LayoutBuilder`。
5. 將回呼參數清單從 `(context)` 修改為 `(context, constraints)`。



每當限制條件變更時，系統就會呼叫 `LayoutBuilder` 的 `builder` 回呼。舉例來說，如果發生下列情況，就會出現這種情形：

- 使用者調整應用程式視窗大小
- 使用者將手機從直向模式轉為橫向模式，或反向操作
- `MyHomePage` 旁邊的小工具變大，導致 `MyHomePage` 的限制變小

現在，您的程式碼可以查詢目前的 `constraints`，決定是否要顯示標籤。對 `_MyHomePageState` 的 `build` 方法進行下列單行變更：

[lib/main.dart](#)

```
// ...

class _MyHomePageState extends State<MyHomePage> {
  var selectedIndex = 0;

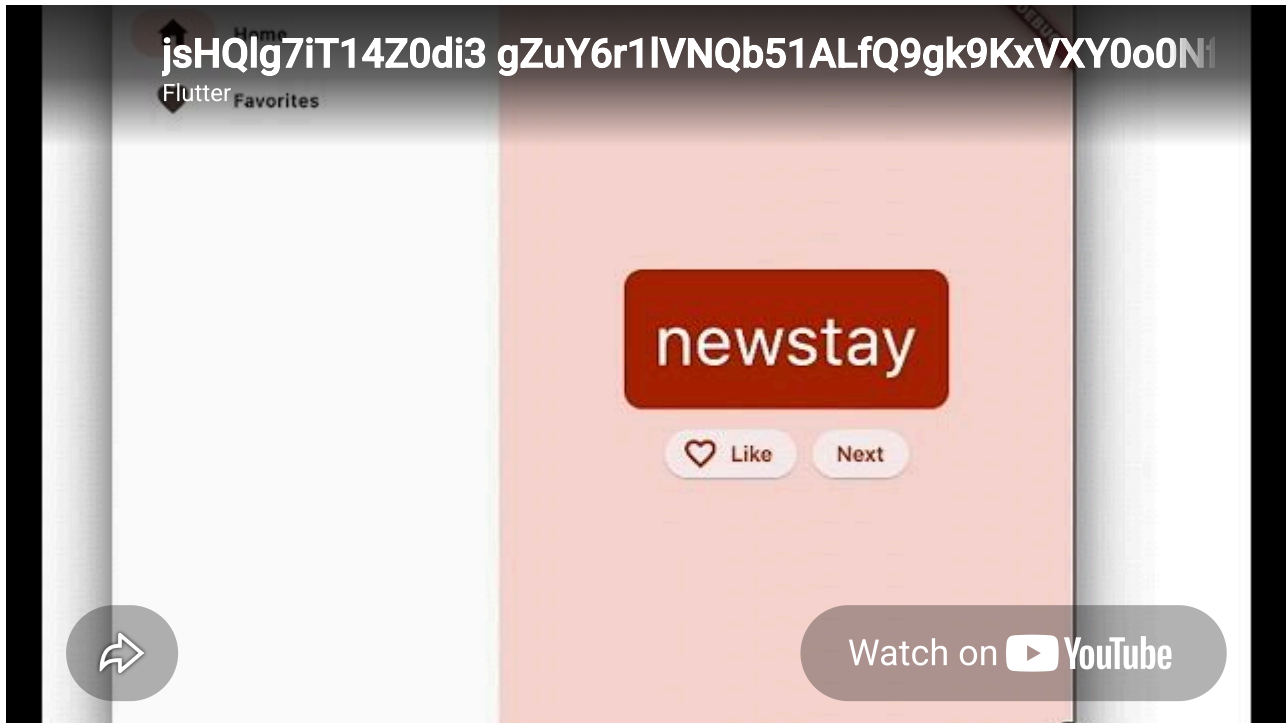
  @override
  Widget build(BuildContext context) {
    Widget page;
    switch (selectedIndex) {
      case 0:
        page = GeneratorPage();
        break;
      case 1:
        page = Placeholder();
        break;
      default:
        throw UnimplementedError('no widget for $selectedIndex');
    }

    return LayoutBuilder(builder: (context, constraints) {
      return Scaffold(
        body: Row(
          children: [
            SafeArea(
              child: NavigationRail(
                extended: constraints.maxWidth >= 600, // ← Here.
                destinations: [
                  NavigationRailDestination(
                    icon: Icon(Icons.home),
                    label: Text('Home'),
                  ),
                  NavigationRailDestination(
                    icon: Icon(Icons.favorite),
                    label: Text('Favorites'),
                  ),
                ],
                selectedIndex: selectedIndex,
                onDestinationSelected: (value) {
                  setState(() {
                    selectedIndex = value;
                  });
                },
              ),
            Expanded(
              child: Container(
                color: Theme.of(context).colorScheme.primaryContainer,
                child: page,
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```



```
);  
});  
}  
}  
  
// ...
```

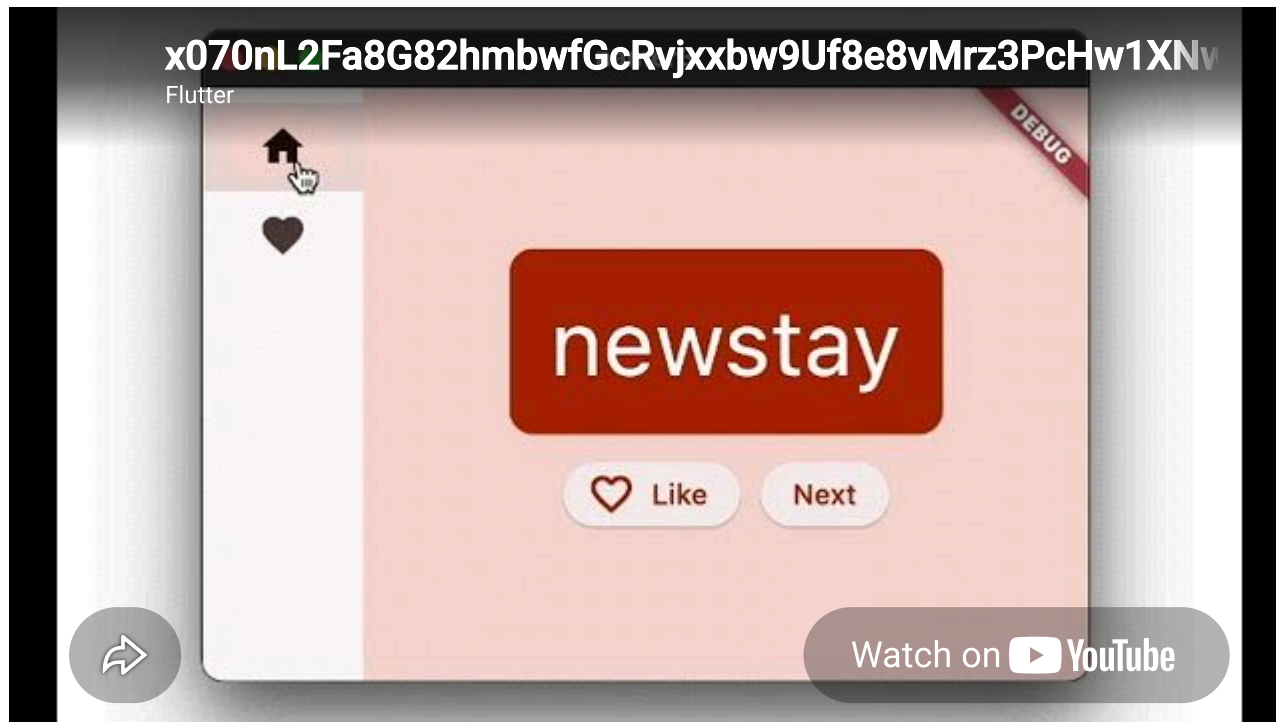
現在，您的應用程式會回應環境變化，例如螢幕大小、方向和平台！換句話說，這項功能會根據情況做出回應！



剩下的工作就是將該 `Placeholder` 替換為實際的「我的最愛」畫面。下一節會說明這項功能。

8. 新增網頁

還記得我們使用 `Placeholder` 小工具，而非「我的最愛」頁面嗎？



現在就來修正這個問題。

如果您想挑戰自我，不妨試著自行完成這個步驟。您的目標是在新的無狀態小工具 `FavoritesPage` 中顯示 `favorites` 清單，然後顯示該小工具，而非 `Placeholder`。

以下提供幾項建議：

- 如要使用可捲動的 `Column`，請使用 `ListView` 小工具。
- 請記住，您可以使用 `context.watch<MyAppState>()` 從任何小工具存取 `MyAppState` 執行個體。
- 如要試用新小工具，`ListTile` 也有 `title` (通常用於文字)、`leading` (用於圖示或虛擬人偶) 和 `onTap` (用於互動) 等屬性。不過，您可以使用已知的 `Widget` 達到類似效果。
- Dart 允許在集合常值中使用 `for` 迴圈。舉例來說，如果 `messages` 包含字串清單，您可以編寫類似下列的程式碼：

```
var messages = ['Hello', 'Ahoj', 'こんにちは'];

return Column(
  children: [
    Text('Messages:'),
    for (var msg in messages)
      Text(msg),
  ],
); // Column
```

另一方面，如果您較熟悉函式程式設計，Dart 也可讓您編寫類似 `messages.map((m) => Text(m)).toList()` 的程式碼。當然，您隨時可以建立小工具清單，並在 `build` 方法中強制加入清單。

自行新增「我的最愛」頁面的優點是，您可以自行做決定，進而瞭解更多資訊。缺點是您可能會遇到自己無法解決的問題。請記住，失敗沒關係，而且是學習過程中最重要的元素之一。沒有人會期待您在第一小時就精通 Flutter 開發，您也不該這樣要求自己。

↓ SPOILERS AHEAD ↓

以下僅是其中一種實作「我的最愛」頁面的方式。希望您能從實作方式獲得靈感，進而修改程式碼，改善 UI 並打造專屬應用程式。

以下是新的 `FavoritesPage` 類別：

[lib/main.dart](#)

```
// ...

class FavoritesPage extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    var appState = context.watch<MyAppState>();

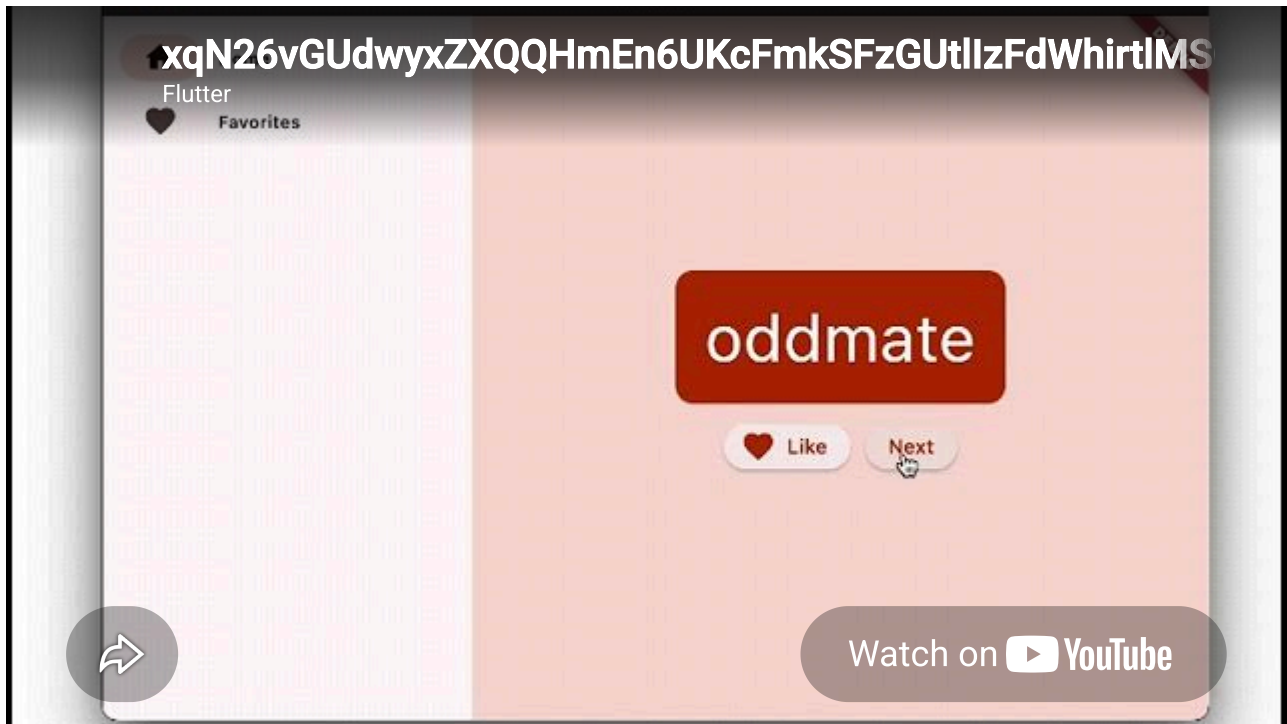
    if (appState.favorites.isEmpty) {
      return Center(
        child: Text('No favorites yet.'),
      );
    }

    return ListView(
      children: [
        Padding(
          padding: const EdgeInsets.all(20),
          child: Text('You have '
            '${appState.favorites.length} favorites:'),
        ),
        for (var pair in appState.favorites)
          ListTile(
            leading: Icon(Icons.favorite),
            title: Text(pair.asLowerCase),
          ),
      ],
    );
  }
}
```

這個小工具的功能如下：

- 取得應用程式的目前狀態。
- 如果收藏清單是空的，畫面中央會顯示「還沒有收藏項目」訊息。
- 否則會顯示 (可捲動的) 清單。
- 清單開頭會顯示摘要 (例如「你已加入 5 個最愛」)。
- 接著，程式碼會逐一查看所有最愛項目，並為每個項目建構 `ListTile` 小工具。

現在只要將 `Placeholder` 小工具換成 `FavoritesPage` 即可。大功告成！



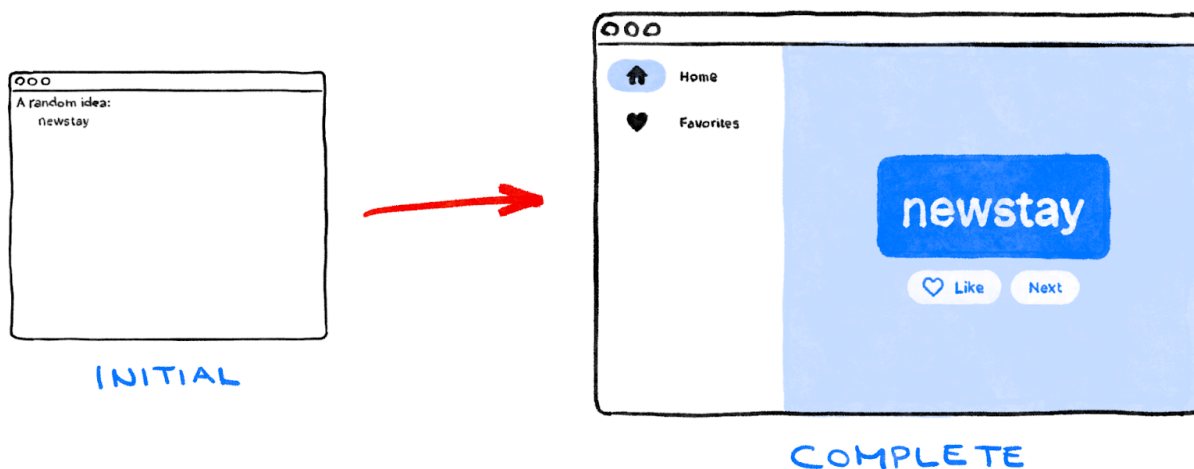
您可以在 GitHub 的 [程式碼研究室存放區](#) 中取得這個應用程式的最終程式碼。

步驟 9 · 約 0 分鐘

9. 後續步驟

恭喜！

你好厲害，您採用了非功能性 Scaffold，其中包含 `Column` 和兩個 `Text` 小工具，並將其改造成回應式的小型應用程式。

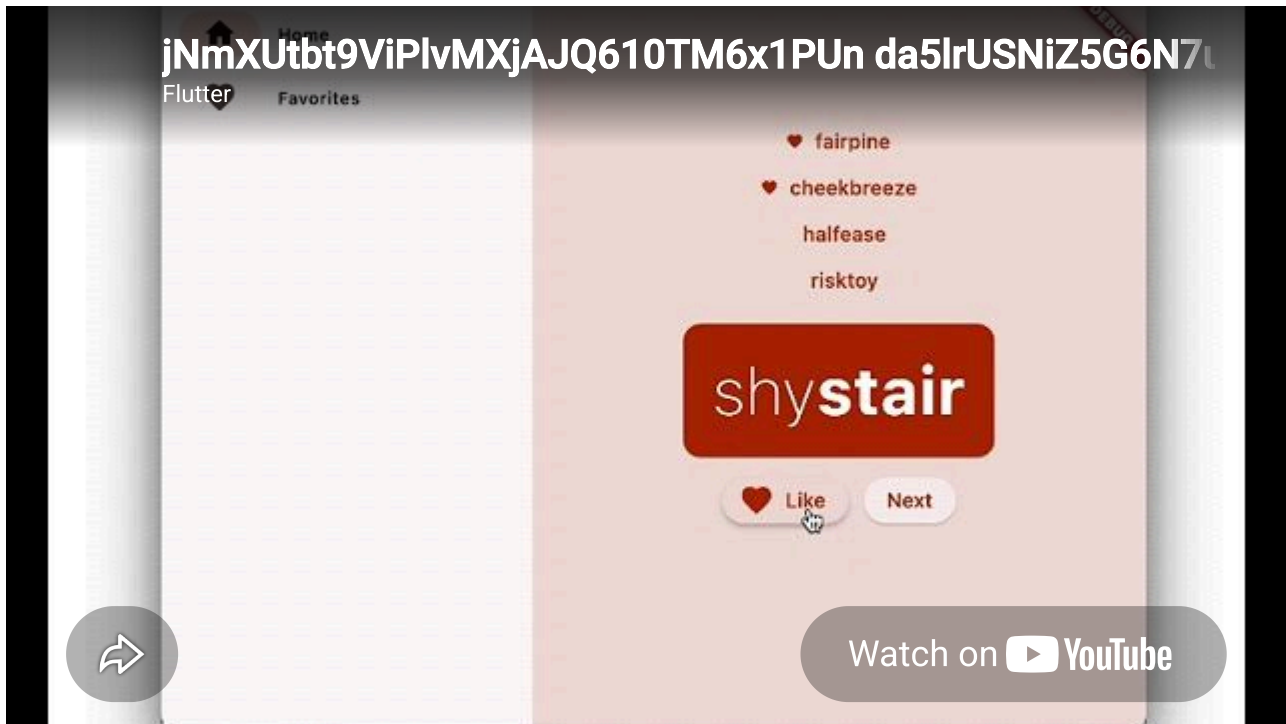


涵蓋內容

- Flutter 的基本運作方式
- 在 Flutter 中建立版面配置
- 將使用者互動 (例如按下按鈕) 連結至應用程式行為
- 讓 Flutter 程式碼井然有序
- 讓應用程式維持正常回應速度
- 打造應用程式一致的外觀和風格

接下來呢？

- 進一步實驗您在本實驗室中編寫的應用程式。
- 請參閱[這個進階版](#)的相同應用程式程式碼，瞭解如何新增動畫清單、漸層、交叉淡化等效果。



- 前往 flutter.dev/learn，繼續學習之旅。